Application of Convex Optimisation in Image Restoration

Pranav Kumar 12495, Thirukovalluru Raghuveer 12762
Department of Electrical Engineering, IIT Kanpur, India
pranavkr@iitk.ac.in, raghuth@iitk.ac.in.

***Abstract*— Image recovery is an important component of most image processing problems. Deburring, In-painting and De-noising form the basic components of this class of image restoration problems. In this report, we review different Image restoration techniques and algorithms used to solve them using some convex optimization techniques. Deblurring, In-Painting and Denoising can be formulated as a convex optimization problems. Multiple Solution techniques like IST, SALSA are used to solve these optimization problems. In this work we discuss in detail some of these techniques and some new formulations for the restoration problems. Specifically, Denoising among the above mentioned three requires to be formulated slightly differently i.e. using an L1 norm for better results. TV-L1 is the typical algorithm used to solve the denoising problem. Experiments are performed to verify the results proposed by different authors and also make comments on the same.**

Keywords—Image restoration; Convex Optimisatioin; Denoising, Debluring and Inpainting; SPARSA

I. INTRODUCTION

IMAGE restoration/reconstruction though being one of the most classical problems of Image processing, continues to attract researchers due to its varied applications in a wide range of problems from astronomy, MRI to cameras in mobile phones. It comprises a class of inverse algorithms applied over an image to improve its quality. Deburring, In-painting and De-noising, Motion-Blurring are some prominent examples of image restoration problems. Most of these problems are solved by means of formulations solved by optimization techniques.

Most of these optimization problems can directly be reformulated and solved as Convex optimization problems. Convex Optimization has hence become a major tool to solve many tasks in Image processing and Computer vision.

We in this work, thoroughly asses some of these tasks and experimentally validate them. The report is further organized as follows. Section II introduces a generalized formulation of image reconstruction problems in mathematical terms. Section III introduces two image reconstruction problems – Deblurring and Inpainting, their formulations and some existing arts solving them. Section IV speaks about another reconstruction problem – Denoising and its adaptation into the scope of the reconstruction formulation. The section also introduces some existing arts solving it. Section IV describes some experimentation and portrays results of the algorithms discussed. Finally Section V draws conclusions from Section IV and comments on some future directions for research.

II. GENERAL FORMULATION

Image restoration is a problem of restoring original images from faulty images. In this class of problems a faulty image can be modelled as

$$y = Bx + n \tag{2.1}$$

B here, is the matrix representation of the modifying process and n is the extra noise added. x is the vector of vertically stacked image pixels. We talk about how B is formulated for different type of distortion very briefly in next section.

This problem of estimating x from y is an ill-posed problem and some regularization has to be adopted to effectively solve it. Following is the objective function with multiple terms catered towards minimizing different types of deviations

$$\min_{\mathrm{x}} \quad \varphi_f(\mathrm{B}x - y) + \varphi_r(Dx) \tag{2.2}$$

B is the blurring matrix which is dependent on the application. The first term in (2) is the "data fidelity" term which forces y to tend to x. φ_f is the penalty function which is typically a squared L2 norm, the L1 norm, or the Huber penalty. The second term in the cost function (2) corresponds to regularization. One key differentiator for natural images is their gradients are sparse in comparison to their unnatural counterpart for the same blurred image. So one possible regularizer technique φ_r is to impose penalty on gradients to ensure sparsity condition. Thus we arrive at one of the most often used regularizer for image restoration, the Total Variation (TV) norm. In this D is a discrete gradient operator and φ_r is L1 norm.

Formulation applies a regularizer directly to the unknown image, leading to criteria of the form

$$\hat{x} = \arg\min_{x} \frac{1}{2} \| \mathrm{Bx} - \mathrm{y} \|_2^2 + \tau\phi(\mathrm{x}) \tag{2.3}$$

III. DEBLURING AND IMPAINTING

A. Problem Formulation

1)Deblurring :

Deblurring is one of the earliest problems of Image Processing. We can classify blur image into primarily into three types:

Camera Shake (blur caused due to movement of hands in the process of taking photograph), Scene Motion (subject in the scene moving), and Defocus (due to incorrect depth of field). In this paper we look at a case where blurring is caused due to handshake. This is modeled by matrix B such that product with it results in all pixel smearing equally into neighboring pixels, mimicking handshake.

2)Inpainting

Parts of image deteriorate with age or due to other processes, for printed photographs it may happen due to folding or fading away of some parts of image, with digital media (image and videos) part of them can be lost in transmission. The purpose of inpainting is to restore these affected portion so they do not appear different from rest of the image. In this paper we look at case when some of the pixels of an image are missing. This is modeled by choosing B in equation (2.3) such that product Bx has some values of element same as corresponding element of x and other as 0 (black/missing).

B. Algorithms

1) Iterative Shrinkage/Thresholding Algorithms

One of the standard algorithm employed to solve problems of the form (2.3) is the iterative shrinkage/thresholding (IST) algorithm. IST can be obtained from a variety of approaches such as forward-backward splitting technique, as an expectation-maximization (EM) algorithm etc. Central function used in IST algorithms is the shrinkage/thresholding function, or the denoising function, corresponding to the regularizer function φ_r , derived by putting B = I in objective function of (2.3). This function $\psi_{\tau\phi} : \mathbb{R}^m \to \mathbb{R}^m$ is defined as:

$$\psi_{\tau\phi}(\mathrm{y}) = \arg\min_x \frac{1}{2}\|\mathrm{x}-\mathrm{y}\|_2^2 + \tau\phi(\mathrm{x}) \qquad (3.1)$$

For, TV based deblurring, $\phi(\mathrm{x})$ is L1 norm ($\phi(\mathrm{x}) = \|x\|_1$), thus the objective function which is being minimized is proper and strictly convex (as sum of two convex functions), therefore we have a unique and existing minimizer. Therefore making the function $\psi_{\tau\phi}$ well defined.

For $\phi(\mathrm{x}) = \|\mathrm{x}\|_1$(TV-based deblurring), the corresponding shrinkage/thresholding function has a known closed form:

$$\psi_{\tau\phi}(\mathrm{y}) = soft(\mathrm{y},\tau) = \begin{cases} y-0.5\tau & if\ \ y>0.5\tau \\ y+0.5\tau & if\ \ y<-0.5\tau \\ 0\ \ otherwise \end{cases} \qquad (3.2)$$

Application of IST for solving (2.3) is given by iterative steps:

$$x_{k+1} = \psi_{\tau\varphi}\left(\mathrm{x}_t - \frac{1}{\gamma}B^H(\mathrm{Bx}_k - \mathrm{y})\right) \qquad (3.3)$$

Here $\frac{1}{\gamma}$ is a step size, $B^H(\mathrm{Bx}_k - \mathrm{y})$ is the gradient of data-fidelity term $B^H(\mathrm{Bx}_k - \mathrm{y})$ at x_k . Therefore each step of IST moves x in negative direction of slope of data fidelity term and operated on by shrinkage/thresholding function corresponding to regularizer function.

It is known that steps of IST converges to a solution of (2.3) given a convex φ and $\gamma > \frac{\|B\|_2^2}{2}$, but it may be quite slow when τ is very small or B is ill-conditioned. Due to this there have been faster variants of IST developed like SpaRSA, two-step IST variants: TwIST and FISTA. Unlike IST in TwIST [3] each step uses two previous values of x rather than just last. We use TwIST for comparison with another method SALSA [1] which is discussed in next paragraph.

2) Split Augmented Lagrangian Shrinkage Algorithm

In SALSA, approach is based on the technique of variable splitting. The objective function of (2.3) to be minimized is the sum of two functions, we split the variable x into a pair of variables, u and v, which serve as the argument for the two functions ($\frac{1}{2}\|\mathrm{Bu}-\mathrm{y}\|_2^2$ & $\tau\phi(\mathrm{v})$), and then the sum is minimized with constraint that both variables are equal, in order that (2.3) and (4.4) are equivalent

$$\begin{aligned} &\min_{x,v\in\mathbb{R}^n} \frac{1}{2}\|\mathrm{Bx}-\mathrm{y}\|_2^2 + \tau\phi(\mathrm{v}) \\ &\text{subject to:} \quad \mathrm{x} = \mathrm{v} \end{aligned} \qquad (3.4)$$

After converting unconstrained optimization (2.3) problem to a constrained optimization problem (3.4) by using variable splitting, this is then addressed with augmented lagrangian (AL) method.

This constrained problem with split variables is solved using algorithm known as alternating direction method of multipliers (ADMM) in which objective function is minimized alternatively with respect to x and v.

After these steps we arrive at following SALSA algorithm:

<u>SALSA ALGORITHM</u>

1. Set k = 0, chose arbitrary v_o , d_o , $\mu > 0$
2. **repeat** till stopping criteria:
   a. $x_{k+1} = \arg\min_x \|\mathrm{B}x - y\|_2^2 + \mu\|x - v_k - d_k\|_2^2$
   b. $v_{k+1} = \arg\min_v \tau\phi(\mathrm{v}) + (\mu/2)\|x_{k+1} - v - d_k\|_2^2$
   c. $d_{k+1} = d_k - (\mathrm{x}_{k+1} - v_{k+1})$
   d. $k \leftarrow k+1$

In step (a) of algorithm objective function to be minimized is strictly convex and we have following formulation for x_{k+1} :

$$x_{k+1} = (\mathrm{B}^H \mathrm{B} + \mu \mathrm{I})^{-1} (\mathrm{B}^H \mathrm{y} + \mu(\mathrm{v}_k + d_k)) \quad (3.5)$$

$\mathrm{B}^H \mathrm{B} + \mu \mathrm{I}$ is a regularized version of Hessian of the data fidelity term of (2.3). Thus SALSA uses second order information of this function whereas variants of IST only used gradient information, this contributes to speed of SALSA over others. We now discuss another image recovery algorithm in section IV.

IV. DENOISING

Denoising is a technique to suppress noise in an image which may have been caused by intrinsic or extrinsic factors which occur in practical situations. A typical Denoising problem can again be adopted into the formulation of Section II as follows.

Given a raw noisy image $f : \Omega \to \mathbb{R}$, where Ω is bounded open subset of $\mathbb{R}^2$, the task at hand here is to obtain an image with suppressed noise such

$$u_\lambda = \arg\min_u \left\{ |u|_p + \lambda \| f - u \|_q^2 \right\} \quad (4.1)$$

$$u_\lambda = \arg\min_u \left\{ J(\mathrm{u}) + H(\mathrm{f}, \mathrm{u}) \right\} \quad (4.2)$$

Here the first term $J(\mathrm{u})$ being the regularization term that fits into $\phi_r(\mathrm{Dx})$ of (2.2) and $H(f, u)$ is the data fidelity term. The presence of both the fidelity term and the regularization term leads to a similar form to that of deblurring typically known as the *ROF model*. Even though, the formulation appears to be similar to that of Deblurring or Inpainting, we can obtain more desirable results with some tweaking of Data-fidelity term. This section discusses the existing algorithms for denoising and the tweaks in objective function for better denoising.

A. ROF Denoising Model

This model was proposed in the early 90's by Rudin-Osher-Fatemi [4] which uses the convexness of the terms of the objective function to propose a solution. The relation, $\partial J(u) = -div(\nabla u / \nabla u)$ can be devised and can be used to arrive at the Eular-Lagrange differential equation as

$$0 = -div(\nabla u / |\nabla u|) + 2\lambda(u - f)$$

Or equivalently

$$u = f + \frac{div(\nabla u | \nabla u)}{2\lambda}$$

The parameter λ determines the denoising level. If λ is small, then it implies that the fidelity term is penalized more than the regularization term and hence, the optimized image u_λ is very close to the original image. On the other hand, for large values of λ, the minimizer highly penalizes the regularization term and the transformed image is different from the original image. Typically, large values of λ result in over-smoothed cartoon like images. Some other models have been developed and were found to suit the denoising problem better than the ROF. TV-L1 is one such algorithm discussed in next section.

B. TV –L1 Denoising Model

TV-L1 as an algorithm is different from ROF model in the fact that it tries and optimizes a different objective function. Typically the L2 norm in (4.1) is replaced by an L1 norm resulting in the following objective function.

$$u_\lambda = \arg\min_u \left\{ |u|_p + \lambda \| f - u \|_q \right\} \quad (4.3)$$

$$\min_x F(Kx) + \sum_i G_i(x) \quad (4.4)$$

Eq. (4.4) is a general notion used to solve the L1 norm problem. This formulation is further modified by dual transformation as (4.5).

$$\min_x \max_p \langle Kx, p \rangle - F^*(p) + \sum_i [\langle x, r_i \rangle - G_i^*(r_i)] \quad (4.6)$$

The iterative updates can then be derived as from [6].

$$p_{n+1} = (I + \sigma \partial F^*)^{-1} (p_n + \sigma K x_n)$$

$$r_{n+1}^i + 1 = (I + \sigma \partial G_i^*)^{-1} (r_n^i + \sigma x_n)$$

$$\hat{x}_{n+1} = x_n - \tau (K^T p_{n+1} + \sum_i r_{n+1}^i)$$

$$x_{n+1} = \hat{x}_{n+1} + \theta (\hat{x}_{n+1} - x_n)$$

Where $(I + \sigma \partial F^*)^{-1}$ is a proximity operator as defined in [6]. p and r are dual variables. This whole procedure of iterative optimization is run for multiple cycles and a denoised image x_{n+1} is obtained.

Experimentally TV-L1 norm is more effective in removal of salt and pepper noise or impulse noise. This being one of the most common type of noise in present day cameras night time photo, the TVL-1 optimization has hence become one of the most widely used denoising techniques available. One another important aspect of this formulation is that it is this is contrast variant i.e. if x is a solution for (4.6), cx is also a solution. The property gives this model a very powerful geometric meaning. This type of model is known to be more effective towards geometric shapes thus increasing its field of application to numerous geometric filtering problems.

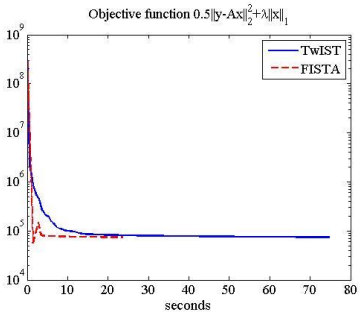
V. EXPERIMENTS

Multiple experiments were performed over all the algorithms discussed above. Following are three sets of experiments pertaining to the three problems discussed.

A. Deblurring

The first set of experiments corresponds to the total variant deblurring. The experiment consists of a uniform blurring operator applied over a clean image as shown in Fig.1 to computationally induce blurring. 40 dB Gaussian noise is then introduced into the image. Finally the SALSA Algorithm as described in III is applied over the noisy and blurry image to get a de-blurred image. Similar procedure is performed with another deblurring algorithm TWIST, a two-step version of IST mentioned in III. We not only compare the deblurred image results of both the algorithms but also the speeds of their execution. We observe that both the deblurred images are almost similar as shown in Fig. 1(c) and Fig.1 (d) as they optimize same cost functions but both algorithms have different running times as can be seen from Fig.1(e), SALSA was much faster than TWIST as can be observed. Quality of restored images remained same.

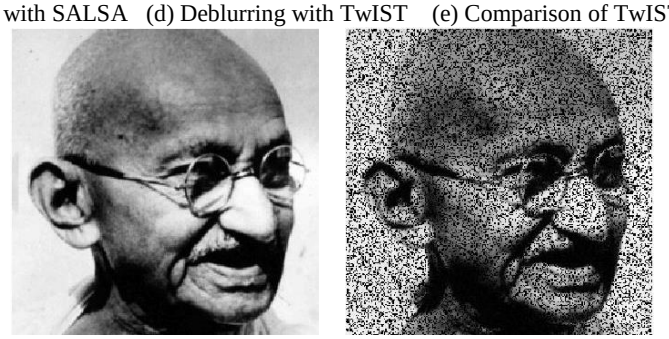
(a) (b)



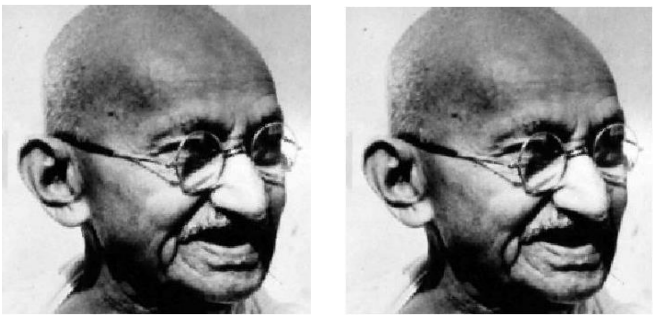
(c) (d) (e)

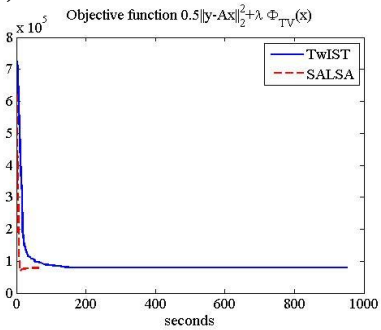
Figure 1: (a) Original Image (b) Blurred Image with noise (SNR = 40 dB)

(c) Deblurring with SALSA (d) Deblurring with TwIST (e) Comparison of TwIST and SALSA



(a) (b)





(c) (d) (e)

Figure 2: (a) Original Image (b) 30% missing pixel image

(c) SALSA based restoration (d) TwIST based restoration (e) SALSA – TwIST comparison

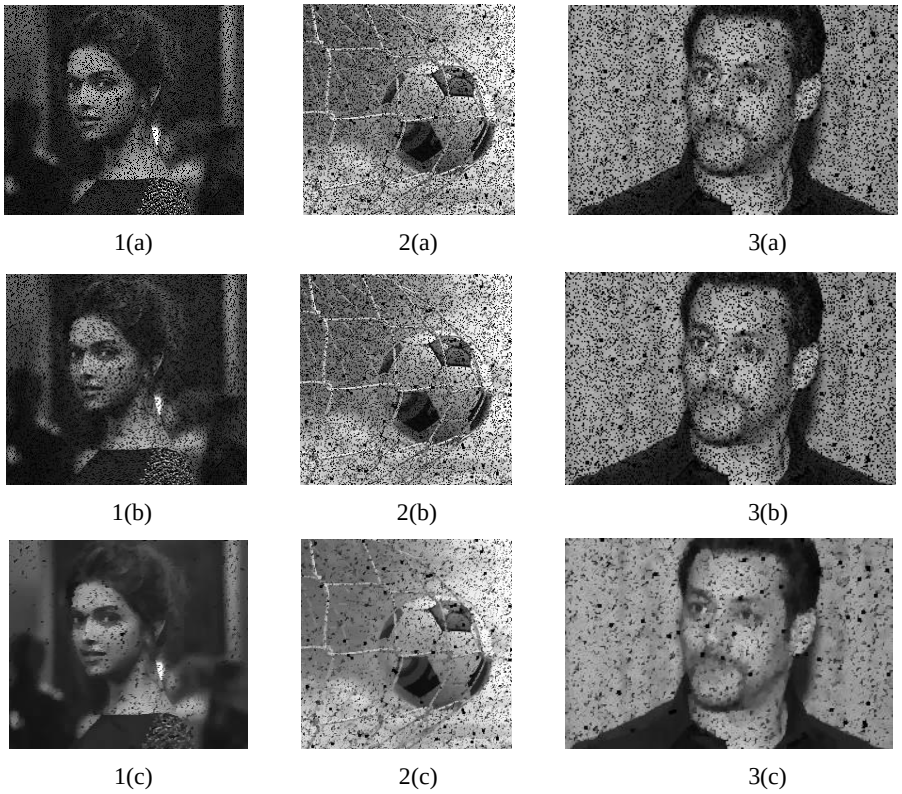


Figure 3: (a) corresponds to 3images with grain noise (b) corresponds to denoising using ROF Model (c) corresponds to denoising with TV-L1 Model

B. Inpainting

The second set of experiments attempts to solve the inpainting problem as described in section III. In these experiments, a clean image as shown in Fig.2(a) is exposed to matrix operation which results in loss of pixels in a randomized manner. SALSA and TWIST algorithms are then imposed over the lossy image to recover pixels. Fig.2(c) and Fig.2(d) correspond to the resultant recovered images form SALSA and TWIST respectively. Here again as shown in Fig. 8 SALSA performed was observed to be much faster than TWIST. The quality of restored images by the two algorithms remained same however.

C. Denoising

The third set of experiments correspond to Denoising. As mentioned, we work with two algorithms in this scope. A clear image is added with high variance gaussian noise. The ROF algorithm and the TV-L1 algorithm are then run over the high noisy image. Fig. 3(b) and Fig. 3(a) correspond to the results of denoised images of ROF and TV-L1 respectively. As, can beseen TVL1 is a better remover of spotty noise/salt pepper noise.

VI. CONCLUSION AND FUTURE WORK

In this work, an exploratory analysis was performed on different Image Restoration techniques. Extensive Experimentation was performed over the different techniques discussed. In both Deblurring and Inpainting – SALSA significantly outperformed TWIST in terms of time although the restored images by both the algorithms were of equal quality when tested with most images. In denoising, it can be concluded that TV-L1 performs slightly better than the than the ROF method in cases of grainy/ impulsive noise in the image. Most of these algorithms use a generalized objective function for all types of images. Further explorations into this field of Image restoration could be use image specific cost functions depending upon the type of image – (Scenery or Close-up or night shot etc.). Machine learning techniques to learn feature properties within the image using CNN/deep networks which can work collaboratively with the optimization techniques discussed is also a very good direction to progress in this field.

VII. REFERENCES

[1] Afonso, Manya V., José M. Bioucas-Dias, and Mário AT Figueiredo. "Fast image recovery using variable splitting and constrained optimization." Image Processing, IEEE Transactions on 19.9 (2010): 2345-2356.

[2] O'Connor, Daniel, and Lieven Vandenberghe. "Total Variation Image Deblurring with Space-Varying Kernel."

[3] Bioucas-Dias, José M., and Mário AT Figueiredo. "A new TwIST: two-step iterative shrinkage/thresholding algorithms for image restoration." Image Processing, IEEE Transactions on 16.12 (2007): 2992-3004.

[4] Rudin, Leonid I., Stanley Osher, and Emad Fatemi. "Nonlinear total variation based noise removal algorithms." Physica D: Nonlinear Phenomena 60.1 (1992): 259-268.

[5] Candes, Emmanuel J., Michael B. Wakin, and Stephen P. Boyd. "Enhancing sparsity by reweighted l1 minimization." Journal of Fourier analysis and applications 14.5-6 (2008): 877-905.

[6] Komodakis, Nikos, and Jean-Christophe Pesquet. "Playing with Duality: An overview of recent primal? dual approaches for solving large-scale optimization problems." Signal Processing Magazine, IEEE 32, no. 6 (2015): 31-54.

[7] Ochs. P; Dosovitskiy. A; Brox. T;Pock. T;"An iterated L1 Algorithm for Non-Smooth Non-Convex Optimization in Computer Vision"IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2013.

[8] Shan, Qi, Jiaya Jia, and Aseem Agarwala. "High-quality motion deblurring from a single image." In *ACM Transactions on Graphics (TOG)*, vol. 27, no. 3, p. 73. ACM, 2008.

VIII. APPENDIX

```
% INPAINTING CODE. Uses TWIST PAckage of codes and Salsa package of codes. Syntaxs  at http://cascais.lx.it.pt/~mafonso/salsa.html

 x = double(imread('cow_2.tif'));
N=length(x);
O = rand(N)> 0.4;
y= x.* O;

% Setting snr
SNR = 50;
P_y = var(x(:));
sigma= sqrt((P_y/10^(SNR/10)));

% Adding the Random Noise
y=y+ sigma*randn(N);
y = y.*O;

% kernel function probably
A = @(x)  O.*x;
AT = @(x) O.*x;
global calls;
calls = 0;
A = @(x) callcounter(A,x);
AT = @(x) callcounter(AT,x);


chambolleit = 20;
Psi_TV = @(x,th)  projk(real(x),th,chambolleit);
Phi_TV = @(x) TVnorm(x);


lambda = 0.2*sigma^2;
mu = 5e-3;
tolA = 1e-4;
outeriters = 500;

invATA = O*(1/(1+mu))+(1-O)*(1/mu);
invLS = @(x) invATA.*x;
invLS = @(x) callcounter(invLS,x);

%%%% SALSA
disp('Running SALSA...\n')
[x_salsa, numA, numAt, objective, distance,  times, mses]= SALSA_v2(y,A,lambda, 'MU', mu, 'AT', AT, 'True_x', x, 'TVINITIALIZATION', 0, 'Psi', Psi_TV,        'Phi', Phi_TV,
'StopCriterion', 1, 'ToleranceA', tolA, 'Initialization',y, 'MAXITERA', outeriters, 'LS', invLS, 'Verbose', 0);
mse_salsa = norm(x- x_salsa,'fro')^2/numel(x);
ISNR_salsa = 10*log10( sum((y(:)-x(:)).^2)./(mse_salsa*numel(x)) );time_salsa = times(end);
 calls_salsa = calls;
calls = 0;

%%%% parameters
lambda = 0.2*sigma^2;
mu = 5e-3;
tolA = 1e-4;
outeriters = 500;


%%%%% -- TwIST --------------------------
disp('Running TwIST...\n')
[x_twist,dummy,obj_twist,
    times_twist,dummy,mses_twist]= TwIST(y,A,lambda,'AT', AT, 'lambda',1e-3,'True_x', x,'Psi', Psi_TV,'Phi',Phi_TV,'Initialization',y,'StopCriterion', 3,'ToleranceA', objective(end),
'MAXITERA', outeriters, 'Verbose', 0);
mse_twist = norm(x- x_twist,'fro')^2/numel(x);
ISNR_twist = 10*log10( sum((y(:)-x(:)).^2)./(mse_twist*numel(x)) );
time_twist = times_twist(end);
calls_twist = calls;
calls = 0;

%%%% display results
figure, colormap(gray), imagesc(x), axis equal, axis off, title('Original');
figure, colormap gray, imagesc(y), axis equal, axis off, title('Missing Samples - 40%');
figure, imagesc(x_salsa), colormap gray, axis equal, axis off, title('Restored Image - SALSA');
figure, imagesc(x_twist), colormap gray, axis equal, axis off, title('Restored Image - TWIST');


figure, plot(times_twist, obj_twist, 'b', 'LineWidth',1.8), hold on,
plot(times, objective,'r--', 'LineWidth',1.8),
title('Objective function 0.5||y-Ax||_{2}^{2}+\lambda \Phi_{TV}(x)','FontName','Times','FontSize',14),
```

```
xlabel('seconds'),legend('TwIST', 'SALSA');


% DEBLURING CODE. Uses TWIST PAckage of codes and Salsa package of codes. Syntax definitions at http://cascais.lx.it.pt/~mafonso/salsa.html

% DEBLURRING CODE. Uses TWIST PAckage of codes and Salsa package of codes. Syntaxs  at http://cascais.lx.it.pt/~mafonso/salsa.html
x = double( imread('sadhu.tif') );
[M, N] = size(x);

%%%% Kernel creation
h = [1 1 1 1 1 1 1 1 1];
lh = length(h);
h = h/sum(h);
h = [h zeros(1,length(x)-length(h))];
h = cshift(h,-(lh-1)/2);
h = h'*h;
H_FFT = fft2(h);
HC_FFT = conj(H_FFT);

A = @(x) real(ifft2(H_FFT.*fft2(x)));
AT = @(x) real(ifft2(HC_FFT.*fft2(x)));

global calls;
calls = 0;
A = @(x) callcounter(A,x);
AT = @(x) callcounter(AT,x);

%%%%% observation
Ax = A(x);
Psig  = norm(Ax,'fro')^2/(M*N);
BSNRdb = 40;
sigma = norm(Ax-mean(mean(Ax)),'fro')/sqrt(N*M*10^(BSNRdb/10));
y = Ax + sigma*randn(M,N);

%%%% algorithm parameters
lambda = 2.5e-2; % reg parameter
mu = lambda/10;
outeriters = 300;
tol = 1e-5;

%%%% TV regularization
Phi_TV = @(x) TVnorm(x);
chambolleit = 5;
Psi_TV = @(x,th)  mex_vartotale(x,th,'itmax',chambolleit);
filter_FFT = 1./(abs(H_FFT).^2 + mu);
invLS = @(x) real(ifft2(filter_FFT.*fft2( x )));
invLS = @(x) callcounter(invLS,x);
 %%%% SALSA
disp('Running SALSA...\n')
[x_salsa, numA, numAt, objective, distance, times, mses] = SALSA_v2(y, A, lambda,'MU', mu,'AT', AT,'StopCriterion', 1,'True_x', x,'ToleranceA', tol,'MAXITERA', outeriters,'Psi',
Psi_TV,'Phi', Phi_TV,'TVINITIALIZATION', 1,'TViters', 10,'LS', invLS,'VERBOSE', 0);
mse = norm(x-x_salsa,'fro')^2 /(M*N);
ISNR = 10*log10( norm(y-x,'fro')^2 / (mse*M*N) );
cpu_time = times(end);

calls_salsa = calls;
calls = 0;
calls_salsa = calls;
calls = 0;
 %%%% TWIST
disp('Running TwIST...\n')
[x_twist,dummy,obj_twist,time_twist,dummy,mse_twist] = TwIST(y, A, lambda, 'AT', AT,'StopCriterion',3, 'True_x', x, 'ToleranceA',objective(end),'PSI', Psi_TV, 'PHI',
Phi_TV,'VERBOSE', 0);
twist_mse = norm(x-x_twist,'fro')^2 /(M*N);
ISNR_twist = 10*log10(  norm(y-x,'fro')^2 / (twist_mse*M*N) );
twist_time = time_twist(end);

calls_twist = calls;
calls = 0;

figure, imagesc(x), title('original'), colormap gray, axis equal, axis off
title('original')
figure, imagesc(y), colormap gray, axis equal, axis off
title('Blurred and noisy')
figure, imagesc(x_salsa), title('Estimated'), colormap gray, axis equal, axis off;
title('Estimated using SALSA')
figure, imagesc(x_twist), title('Estimated'), colormap gray, axis equal, axis off;
title('Estimated using TWIST')

figure, semilogy(time_twist, obj_twist, 'b', 'LineWidth',1.8), hold on,
semilogy(times, objective,'r--', 'LineWidth',1.8),
title('Objective function ','FontName','Times','FontSize',14),
```

```
xlabel('seconds'),
legend('TwIST', 'SALSA');


import numpy as np
%%%%DENOISING CODE  Syntaxes  at http://znah.net/rof-and-tv-l1-denoising-with-primal-dual-algorithm.html


import numpy
from nbutils import showarray
from scipy.misc import lena
import cv2

img_ref = cv2.imread('ball.jpg',0);
showarray(img_ref);
#img_ref = img_ref[0:,0:][:256,:256] / 255.0;


def solve_ROF(img, clambda, iter_n=10001):
    # setting step sizes and other params
    L2 = 1.0
    tau = 0.02
    sigma = 1.0 / (L2*tau)
    theta = 1.0
    X = img.copy()
    P = nabla(X)
    for i in xrange(iter_n):
        P = project_nd( P + sigma*nabla(X), 1.0 )
        lt = clambda * tau
        X1 = (X - tau * nablaT(P) + lt * img) / (1.0 + lt)
        X = X1 + theta * (X1 - X)
        if i % 10 == 0:
            print "%.2f" % calc_energy_ROF(X, img, clambda),
    print
    return X
img_rof = solve_ROF(observations[0], 1.0);
showarray(img_rof);
cv2.imwrite('./res/rof_fb.png',img_rof);

def solve_TVL1(img, clambda, iter_n=1501):
    # setting step sizes and other params
    L2 = 1.0
    tau = 0.02
    sigma = 1.0 / (L2*tau)
    theta = 1.0
    X = img.copy()
    P = nabla(X)
    for i in xrange(iter_n):
        P = project_nd( P + sigma*nabla(X), 1.0 )
        X1 = shrink_1d(X - tau*nablaT(P), img, clambda*tau)
        X = X1 + theta * (X1 - X)
        if i % 10 == 0:
            print "%.2f" % calc_energy_TVL1(X, img, clambda),
    print
    return X

img_tvl1 = solve_TVL1(observations[0], 1.0);
showarray(img_tvl1);
cv2.imwrite('./res/tv_fb.png',img_tvl1);
```

FUNCTIONS @ http://znah.net/rof-and-tv-l1-denoising-with-primal-dual-algorithm.html

```
def make_noisy(img):
    # add gaussian noise
    img = img + 0.1 * np.random.normal(size=img.shape)
    # add some outliers in on the right side of the image
    m = np.random.rand(*img.shape) < 0.2
    #m[:,:160] = 0
    img[m] = np.random.rand(m.sum())
    return img

def nabla(I):
    h, w = I.shape
    G = np.zeros((h, w, 2), I.dtype)
    G[:, :-1, 0] -= I[:, :-1]
    G[:, :-1, 0] += I[:, 1:]
    G[:-1, :, 1] -= I[:-1]
    G[:-1, :, 1] += I[1:]
    return G

def nablaT(G):
    h, w = G.shape[:2]
    I = np.zeros((h, w), G.dtype)
    # note that we just reversed left and right sides
    # of each line to obtain the transposed operator
    I[:, :-1] -= G[:, :-1, 0]
    I[:, 1: ] += G[:, :-1, 0]
    I[:-1]    -= G[:-1, :, 1]
    I[1: ]    += G[:-1, :, 1]
    return I

def anorm(x):
    '''Calculate L2 norm over the last array dimention'''
    return np.sqrt((x*x).sum(-1))

def calc_energy_ROF(X, observation, clambda):
    Ereg = anorm(nabla(X)).sum()
    Edata = 0.5 * clambda * ((X - observation)**2).sum()
    return Ereg + Edata

def calc_energy_TVL1(X, observation, clambda):
    Ereg = anorm(nabla(X)).sum()
    Edata = clambda * np.abs(X - observation).sum()
    return Ereg + Edata

def project_nd(P, r):
    '''perform a pixel-wise projection onto R-radius balls'''
    nP = np.maximum(1.0, anorm(P)/r)
    return P / nP[...,np.newaxis]

def shrink_1d(X, F, step):
    '''pixel-wise scalar srinking'''
    return X + np.clip(F - X, -step, step)

def make_spotty(img, r=3, n=50):
    img = img.copy()
    h, w = img.shape
    for i in xrange(n):
        x, y = np.int16(np.random.rand(2)*(w-r, h-r))
        img[y:y+r, x:x+r] = round(np.random.rand())
    return img

observations = [make_spotty(make_noisy(img_ref)) for i in xrange(5)]
showarray(observations[0])
cv2.imwrite('./res/noise_fb.png',observations[0]);
```

TWIST basic functions

……..

```
function [x,x_debias,objective,times,debias_start,mses,max_svd] = ...
      TwIST(y,A,tau,varargin)
%
% Usage:
% [x,x_debias,objective,times,debias_start,mses] = TwIST(y,A,tau,varargin)
%
% This function solves the regularization problem
%
%     arg min_x = 0.5*|| y - A x ||_2^2 + tau phi( x ),
%
% where A is a generic matrix and phi(.) is a regularizarion
% function  such that the solution of the denoising problem
%
%     Psi_tau(y) = arg min_x = 0.5*|| y - x ||_2^2 + tau \phi( x ),
%
% is known.
%
% For further details about the TwIST algorithm, see the paper:
%
% J. Bioucas-Dias and M. Figueiredo, "A New TwIST: Two-Step
% Iterative Shrinkage/Thresholding Algorithms for Image
% Restoration",  IEEE Transactions on Image processing, 2007.
%
% and
%
% J. Bioucas-Dias and M. Figueiredo, "A Monotonic Two-Step
% Algorithm for Compressive Sensing and Other Ill-Posed
% Inverse Problems", submitted, 2007.
%
% Authors: Jose Bioucas-Dias and Mario Figueiredo, October, 2007.
%
% Please check for the latest version of the code and papers at
% www.lx.it.pt/~bioucas/TwIST
%
% ----------------------------------------------------------------------------
% Copyright (2007): Jose Bioucas-Dias and Mario Figueiredo
%
% TwIST is distributed under the terms of
% the GNU General Public License 2.0.
%
% Permission to use, copy, modify, and distribute this software for
% any purpose without fee is hereby granted, provided that this entire
% notice is included in all copies of any software which is or includes
% a copy or modification of this software and in all copies of the
% supporting documentation for such software.
% This software is being provided "as is", without any express or
% implied warranty.  In particular, the authors do not make any
% representation or warranty of any kind concerning the merchantability
% of this software or its fitness for any particular purpose."
% ----------------------------------------------------------------------------
%
%  ===== Required inputs =============
%
%  y: 1D vector or 2D array (image) of observations
%
%  A: if y and x are both 1D vectors, A can be a
%     k*n (where k is the size of y and n the size of x)
%     matrix or a handle to a function that computes
%     products of the form A*v, for some vector v.
%     In any other case (if y and/or x are 2D arrays),
%     A has to be passed as a handle to a function which computes
%     products of the form A*x; another handle to a function
%     AT which computes products of the form A'*x is also required
%     in this case. The size of x is determined as the size
%     of the result of applying AT.
%
%  tau: regularization parameter, usually a non-negative real
%       parameter of the objective  function (see above).
%
%
%  ===== Optional inputs =============
%
%  'Psi' = denoising function handle; handle to denoising function
%          Default = soft threshold.
%
%  'Phi' = function handle to regularizer needed to compute the objective
%          function.
%          Default = ||x||_1
%
%  'lambda' = lam1 parameters of the  TwIST algorithm:
%             Optimal choice: lam1 = min eigenvalue of A'*A.
%             If min eigenvalue of A'*A == 0, or unknwon,
%             set lam1 to a value much smaller than 1.
%
%             Rule of Thumb:
%                lam1=1e-4 for severyly ill-conditioned problems
%                lam1=1e-2 for mildly  ill-conditioned problems
%                lam1=1    for A unitary direct operators
%
%             Default: lam1 = 0.04.
%
%             Important Note: If (max eigenvalue of A'*A) > 1,
%             the algorithm may diverge. This is  be avoided
%             by taking one of the follwoing  measures:
%
%               1) Set 'Monontone' = 1 (default)
%
%               2) Solve the equivalenve minimization problem
%
%             min_x = 0.5*|| (y/c) - (A/c) x ||_2^2 + (tau/c^2) \phi( x ),
%
%             where c > 0 ensures that  max eigenvalue of (A'A/c^2) <= 1.
```

```
%
%  'alpha' = parameter alpha of TwIST (see ex. (22) of the paper)
%           Default alpha = alpha(lamN=1, lam1)
%
%  'beta'  =  parameter beta of twist (see ex. (23) of the paper)
%            Default beta = beta(lamN=1, lam1)
%
% 'AT'    = function handle for the function that implements
%          the multiplication by the conjugate of A, when A
%          is a function handle.
%          If A is an array, AT is ignored.
%
% 'StopCriterion' = type of stopping criterion to use
%                 0 = algorithm stops when the relative
%                     change in the number of non-zero
%                     components of the estimate falls
%                     below 'ToleranceA'
%                 1 = stop when the relative
%                     change in the objective function
%                     falls below 'ToleranceA'
%                 2 = stop when the relative norm of the difference between
%                     two consecutive estimates falls below toleranceA
%                 3 = stop when the objective function
%                     becomes equal or less than toleranceA.
%                 Default = 1.
%
% 'ToleranceA' = stopping threshold; Default = 0.01
%
% 'Debias'     = debiasing option: 1 = yes, 0 = no.
%              Default = 0.
%
%              Note: Debiasing is an operation aimed at the
%              computing the solution of the LS problem
%
%                    arg min_x = 0.5*|| y - A' x' ||_2^2
%
%              where A' is the  submatrix of A obatained by
%              deleting the columns of A corresponding of components
%              of x set to zero by the TwIST algorithm
%
%
% 'ToleranceD' = stopping threshold for the debiasing phase:
%              Default = 0.0001.
%              If no debiasing takes place, this parameter,
%              if present, is ignored.
%
% 'MaxiterA' = maximum number of iterations allowed in the
%             main phase of the algorithm.
%             Default = 1000
%
% 'MiniterA' = minimum number of iterations performed in the
%             main phase of the algorithm.
%             Default = 5
%
% 'MaxiterD' = maximum number of iterations allowed in the
%             debising phase of the algorithm.
%             Default = 200
%
% 'MiniterD' = minimum number of iterations to perform in the
%             debiasing phase of the algorithm.
%             Default = 5
%
% 'Initialization' must be one of {0,1,2,array}
%             0 -> Initialization at zero.
%             1 -> Random initialization.
%             2 -> initialization with A'*y.
%             array -> initialization provided by the user.
%             Default = 0;
%
% 'Monotone' = enforce monotonic decrease in f.
%             any nonzero -> enforce monotonicity
%             0 -> don't enforce monotonicity.
%             Default = 1;
%
% 'Sparse'   = {0,1} accelarates the convergence rate when the regularizer
%             Phi(x) is sparse inducing, such as ||x||_1.
%             Default = 1
%
%
% 'True_x' = if the true underlying x is passed in
%              this argument, MSE evolution is computed
%
%
% 'Verbose'  = work silently (0) or verbosely (1)
%
% ==================================================
% ========== Outputs ==============================
%   x = solution of the main algorithm
%
%   x_debias = solution after the debiasing phase;
%                if no debiasing phase took place, this
%                variable is empty, x_debias = [].
%
%   objective = sequence of values of the objective function
%
%   times = CPU time after each iteration
%
%   debias_start = iteration number at which the debiasing
%                 phase started. If no debiasing took place,
%                 this variable is returned as zero.
%
%   mses = sequence of MSE values, with respect to True_x,
%         if it was given; if it was not given, mses is empty,
%         mses = [].
%
%   max_svd = inverse of the scaling factor, determined by TwIST,
%            applied to the direct operator (A/max_svd) such that
%            every IST step is increasing.
% ========================================================

%-----------------------------------------------------------------
% test for number of required parametres
%-----------------------------------------------------------------
if (nargin-length(varargin)) ~= 3
    error('Wrong number of required parameters');
```

```
end
%-----------------------------------------------------------------
% Set the defaults for the optional parameters
%-----------------------------------------------------------------
stopCriterion = 1;
tolA = 0.01;
debias = 0;
maxiter = 1000;
maxiter_debias = 200;
miniter = 5;
miniter_debias = 5;
init = 0;
enforceMonotone = 1;
compute_mse = 0;
plot_ISNR = 0;
AT = 0;
verbose = 1;
alpha = 0;
beta  = 0;
sparse = 1;
tolD = 0.001;
phi_l1 = 0;
psi_ok = 0;
% default eigenvalues
lam1=1e-4;   lamN=1;
%

% constants ans internal variables
for_ever = 1;
% maj_max_sv: majorizer for the maximum singular value of operator A
max_svd = 1;

% Set the defaults for outputs that may not be computed
debias_start = 0;
x_debias = [];
mses = [];

%-----------------------------------------------------------------
% Read the optional parameters
%-----------------------------------------------------------------
if (rem(length(varargin),2)==1)
  error('Optional parameters should always go by pairs');
else
  for i=1:2:(length(varargin)-1)
    switch upper(varargin{i})
     case 'LAMBDA'
       lam1 = varargin{i+1};
     case 'ALPHA'
       alpha = varargin{i+1};
     case 'BETA'
       beta = varargin{i+1};
     case 'PSI'
       psi_function = varargin{i+1};
     case 'PHI'
       phi_function = varargin{i+1};
     case 'STOPCRITERION'
       stopCriterion = varargin{i+1};
     case 'TOLERANCEA'
       tolA = varargin{i+1};
     case 'TOLERANCED'
       tolD = varargin{i+1};
     case 'DEBIAS'
       debias = varargin{i+1};
     case 'MAXITERA'
       maxiter = varargin{i+1};
     case 'MAXIRERD'
       maxiter_debias = varargin{i+1};
     case 'MINITERA'
       miniter = varargin{i+1};
     case 'MINITERD'
       miniter_debias = varargin{i+1};
     case 'INITIALIZATION'
       if prod(size(varargin{i+1})) > 1   % we have an initial x
                        init = 33333;    % some flag to be used below
                        x = varargin{i+1};
       else
                        init = varargin{i+1};
       end
     case 'MONOTONE'
       enforceMonotone = varargin{i+1};
     case 'SPARSE'
       sparse = varargin{i+1};
     case 'TRUE_X'
       compute_mse = 1;
       true = varargin{i+1};
       if prod(double((size(true) == size(y))))
          plot_ISNR = 1;
       end
     case 'AT'
       AT = varargin{i+1};
     case 'VERBOSE'
       verbose = varargin{i+1};
     otherwise
      % Hmmm, something wrong with the parameter string
      error(['Unrecognized option: ''' varargin{i} '''']);
    end;
  end;
end
%%%%%%%%%%%%%%%


% twist parameters
rho0 = (1-lam1/lamN)/(1+lam1/lamN);
if alpha == 0
    alpha = 2/(1+sqrt(1-rho0^2));
end
if  beta == 0
    beta  = alpha*2/(lam1+lamN);
end


if (sum(stopCriterion == [0 1 2 3])==0)
   error(['Unknwon stopping criterion']);
end
```

```
% if A is a function handle, we have to check presence of AT,
if isa(A, 'function_handle') & ~isa(AT,'function_handle')
   error(['The function handle for transpose of A is missing']);
end


% if A is a matrix, we find out dimensions of y and x,
% and create function handles for multiplication by A and A',
% so that the code below doesn't have to distinguish between
% the handle/not-handle cases
if ~isa(A, 'function_handle')
   AT = @(x) A'*x;
   A = @(x) A*x;
end
% from this point down, A and AT are always function handles.

% Precompute A'*y since it'll be used a lot
Aty = AT(y);

% if phi was given, check to see if it is a handle and that it
% accepts two arguments
if exist('psi_function','var')
   if isa(psi_function,'function_handle')
      try  % check if phi can be used, using Aty, which we know has
           % same size as x
            dummy = psi_function(Aty,tau);
            psi_ok = 1;
      catch
         error(['Something is wrong with function handle for psi'])
      end
   else
      error(['Psi does not seem to be a valid function handle']);
   end
else %if nothing was given, use soft thresholding
   psi_function = @(x,tau) soft(x,tau);
end

% if psi exists, phi must also exist
if (psi_ok == 1)
   if exist('phi_function','var')
      if isa(phi_function,'function_handle')
         try  % check if phi can be used, using Aty, which we know has
              % same size as x
              dummy = phi_function(Aty);
         catch
           error(['Something is wrong with function handle for phi'])
         end
      else
        error(['Phi does not seem to be a valid function handle']);
      end
   else
      error(['If you give Psi you must also give Phi']);
   end
else  % if no psi and phi were given, simply use the l1 norm.
   phi_function = @(x) sum(abs(x(:)));
   phi_l1 = 1;
end


%-----------------------------------------------------------------
% Initialization
%-----------------------------------------------------------------
switch init
    case 0   % initialize at zero, using AT to find the size of x
       x = AT(zeros(size(y)));
    case 1   % initialize randomly, using AT to find the size of x
       x = randn(size(AT(zeros(size(y)))));
    case 2   % initialize x0 = A'*y
       x = Aty;
    case 33333
       % initial x was given as a function argument; just check size
       if size(A(x)) ~= size(y)
          error(['Size of initial x is not compatible with A']);
       end
    otherwise
       error(['Unknown "Initialization" option']);
end

% now check if tau is an array; if it is, it has to
% have the same size as x
if prod(size(tau)) > 1
   try,
      dummy = x.*tau;
   catch,
      error(['Parameter tau has wrong dimensions; it should be scalar or size(x)']),
   end
end

% if the true x was given, check its size
if compute_mse & (size(true) ~= size(x))
   error(['Initial x has incompatible size']);
end


% if tau is large enough, in the case of phi = l1, thus psi = soft,
% the optimal solution is the zero vector
if phi_l1
   max_tau = max(abs(Aty(:)));
   if (tau >= max_tau)&(psi_ok==0)
      x = zeros(size(Aty));
      objective(1) = 0.5*(y(:)'*y(:));
      times(1) = 0;
      if compute_mse
        mses(1) = sum(true(:).^2);
      end
      return
   end
end


% define the indicator vector or matrix of nonzeros in x
nz_x = (x ~= 0.0);
num_nz_x = sum(nz_x(:));

% Compute and store initial value of the objective function
```

```
resid =  y-A(x);
prev_f = 0.5*(resid(:)'*resid(:)) + tau*phi_function(x);


% start the clock
t0 = cputime;

times(1) = cputime - t0;
objective(1) = prev_f;

if compute_mse
   mses(1) = sum(sum((x-true).^2));
end

cont_outer = 1;
iter = 1;

if verbose
    fprintf(1,'\nInitial objective = %10.6e,  nonzeros=%7d\n',...
        prev_f,num_nz_x);
end

% variables controling first and second order iterations
IST_iters = 0;
TwIST_iters = 0;

% initialize
xm2=x;
xm1=x;

%-----------------------------------------------------------------
% TwIST iterations
%-----------------------------------------------------------------
while cont_outer
    % gradient
    grad = AT(resid);
    while for_ever
        % IST estimate
        x = psi_function(xm1 + grad/max_svd,tau/max_svd);
        if (IST_iters >= 2) | ( TwIST_iters ~= 0)
            % set to zero the past when the present is zero
            % suitable for sparse inducing priors
            if sparse
                mask = (x ~= 0);
                xm1 = xm1.* mask;
                xm2 = xm2.* mask;
            end
            % two-step iteration
            xm2 = (alpha-beta)*xm1 + (1-alpha)*xm2 + beta*x;
            % compute residual
            resid = y-A(xm2);
            f = 0.5*(resid(:)'*resid(:)) + tau*phi_function(xm2);
            if (f > prev_f) & (enforceMonotone)
                TwIST_iters = 0;  % do a IST iteration if monotonocity fails
            else
                TwIST_iters = TwIST_iters+1; % TwIST iterations
                IST_iters = 0;
                x = xm2;
                if mod(TwIST_iters,10000) == 0
                    max_svd = 0.9*max_svd;
                end
                break;  % break loop while
            end
        else
            resid = y-A(x);
            f = 0.5*(resid(:)'*resid(:)) + tau*phi_function(x);
            if f > prev_f
                % if monotonicity  fails here  is  because
                % max eig (A'A) > 1. Thus, we increase our guess
                % of max_svs
                max_svd = 2*max_svd;
                if verbose
                    fprintf('Incrementing S=%2.2e\n',max_svd)
                end
                IST_iters = 0;
                TwIST_iters = 0;
            else
                TwIST_iters = TwIST_iters + 1;
                break;  % break loop while
            end
        end
    end

    xm2 = xm1;
    xm1 = x;



    %update the number of nonzero components and its variation
    nz_x_prev = nz_x;
    nz_x = (x~=0.0);
    num_nz_x = sum(nz_x(:));
    num_changes_active = (sum(nz_x(:)~=nz_x_prev(:)));

    % take no less than miniter and no more than maxiter iterations
    switch stopCriterion
        case 0,
            % compute the stopping criterion based on the change
            % of the number of non-zero components of the estimate
            criterion =  num_changes_active;
        case 1,
            % compute the stopping criterion based on the relative
            % variation of the objective function.
            criterion = abs(f-prev_f)/prev_f;
        case 2,
            % compute the stopping criterion based on the relative
            % variation of the estimate.
            criterion = (norm(x(:)-xm1(:))/norm(x(:)));
        case 3,
            % continue if not yet reached target value tolA
            criterion = f;
        otherwise,
            error(['Unknwon stopping criterion']);
    end
    cont_outer = ((iter <= maxiter) & (criterion > tolA));
```

```
    if iter <= miniter
        cont_outer = 1;
    end



    iter = iter + 1;
    prev_f = f;
    objective(iter) = f;
    times(iter) = cputime-t0;

    if compute_mse
        err = true - x;
        mses(iter) = (err(:)'*err(:));
    end

    % print out the various stopping criteria
    if verbose
        if plot_ISNR
            fprintf(1,'Iteration=%4d, ISNR=%4.5e  objective=%9.5e, nz=%7d, criterion=%7.3e\n',...
                iter, 10*log10(sum((y(:)-true(:)).^2)/sum((x(:)-true(:)).^2) ), ...
                f, num_nz_x, criterion/tolA);
        else
            fprintf(1,'Iteration=%4d, objective=%9.5e, nz=%7d,  criterion=%7.3e\n',...
                iter, f, num_nz_x, criterion/tolA);
        end
    end

end
%-----------------------------------------------------------------
% end of the main loop
%-----------------------------------------------------------------

% Printout results
if verbose
    fprintf(1,'\nFinished the main algorithm!\nResults:\n')
    fprintf(1,'||A x - y ||_2 = %10.3e\n',resid(:)'*resid(:))
    fprintf(1,'||x||_1 = %10.3e\n',sum(abs(x(:))))
    fprintf(1,'Objective function = %10.3e\n',f);
    fprintf(1,'Number of non-zero components = %d\n',num_nz_x);
    fprintf(1,'CPU time so far = %10.3e\n', times(iter));
    fprintf(1,'\n');
end


%-----------------------------------------------------------------
% If the 'Debias' option is set to 1, we try to
% remove the bias from the l1 penalty, by applying CG to the
% least-squares problem obtained by omitting the l1 term
% and fixing the zero coefficients at zero.
%-----------------------------------------------------------------
if debias
    if verbose
        fprintf(1,'\n')
        fprintf(1,'Starting the debiasing phase...\n\n')
    end

    x_debias = x;
    zeroind = (x_debias~=0);
    cont_debias_cg = 1;
    debias_start = iter;

    % calculate initial residual
    resid = A(x_debias);
    resid = resid-y;
    resid_prev = eps*ones(size(resid));

    rvec = AT(resid);

    % mask out the zeros
    rvec = rvec .* zeroind;
    rTr_cg = rvec(:)'*rvec(:);

    % set convergence threshold for the residual || RW x_debias - y ||_2
    tol_debias = tolD * (rvec(:)'*rvec(:));

    % initialize pvec
    pvec = -rvec;

    % main loop
    while cont_debias_cg

        % calculate A*p = Wt * Rt * R * W * pvec
        RWpvec = A(pvec);
        Apvec = AT(RWpvec);

        % mask out the zero terms
        Apvec = Apvec .* zeroind;

        % calculate alpha for CG
        alpha_cg = rTr_cg / (pvec(:)'* Apvec(:));

        % take the step
        x_debias = x_debias + alpha_cg * pvec;
        resid = resid + alpha_cg * RWpvec;
        rvec  = rvec  + alpha_cg * Apvec;

        rTr_cg_plus = rvec(:)'*rvec(:);
        beta_cg = rTr_cg_plus / rTr_cg;
        pvec = -rvec + beta_cg * pvec;

        rTr_cg = rTr_cg_plus;

        iter = iter+1;

        objective(iter) = 0.5*(resid(:)'*resid(:)) + ...
            tau*phi_function(x_debias(:));
        times(iter) = cputime - t0;

        if compute_mse
            err = true - x_debias;
            mses(iter) = (err(:)'*err(:));
        end

        % in the debiasing CG phase, always use convergence criterion
```

```
        % based on the residual (this is standard for CG)
        if verbose
            fprintf(1,' Iter = %5d, debias resid = %13.8e, convergence = %8.3e\n', ...
                iter, resid(:)'*resid(:), rTr_cg / tol_debias);
        end
        cont_debias_cg = ...
            (iter-debias_start <= miniter_debias )| ...
            ((rTr_cg > tol_debias) & ...
            (iter-debias_start <= maxiter_debias));

    end
    if verbose
        fprintf(1,'\nFinished the debiasing phase!\nResults:\n')
        fprintf(1,'||A x - y ||_2 = %10.3e\n',resid(:)'*resid(:))
        fprintf(1,'||x||_1 = %10.3e\n',sum(abs(x(:))))
        fprintf(1,'Objective function = %10.3e\n',f);
        nz = (x_debias~=0.0);
        fprintf(1,'Number of non-zero components = %d\n',sum(nz(:)));
        fprintf(1,'CPU time so far = %10.3e\n', times(iter));
        fprintf(1,'\n');
    end
end

if compute_mse
    mses = mses/length(true(:));
end


%-----------------------------------------------------------------
% soft for both real and  complex numbers
%-----------------------------------------------------------------
function y = soft(x,T)
%y = sign(x).*max(abs(x)-tau,0);
y = max(abs(x) - T, 0);
y = y./(y+T) .* x;





function y = conv2c(x,h)
% Circular 2D convolution
x=wraparound(x,h);
y=conv2(x,h,'valid');


function y = wraparound(x, m)
% Extend x so as to wrap around on both axes, sufficient to allow a
% "valid" convolution with m to return the cyclical convolution.
% We assume mask origin near centre of mask for compatibility with
% "same" option.
[mx, nx] = size(x);
[mm, nm] = size(m);
if mm > mx | nm > nx
    error('Mask does not fit inside array')
end

mo = floor((1+mm)/2); no = floor((1+nm)/2);  % reflected mask origin
ml = mo-1;            nl = no-1;             % mask left/above origin
mr = mm-mo;           nr = nm-no;            % mask right/below origin
me = mx-ml+1;         ne = nx-nl+1;          % reflected margin in input
mt = mx+ml;           nt = nx+nl;            % top of image in output
my = mx+mm-1;         ny = nx+nm-1;          % output size

y = zeros(my, ny);
y(mo:mt, no:nt) = x;      % central region
if ml > 0
    y(1:ml, no:nt) = x(me:mx, :);                % top side
    if nl > 0
        y(1:ml, 1:nl) = x(me:mx, ne:nx);         % top left corner
    end
    if nr > 0
        y(1:ml, nt+1:ny) = x(me:mx, 1:nr);       % top right corner
    end
end
if mr > 0
    y(mt+1:my, no:nt) = x(1:mr, :);              % bottom side
    if nl > 0
        y(mt+1:my, 1:nl) = x(1:mr, ne:nx);       % bottom left corner
    end
    if nr > 0
        y(mt+1:my, nt+1:ny) = x(1:mr, 1:nr);     % bottom right corner
    end
end
if nl > 0
    y(mo:mt, 1:nl) = x(:, ne:nx);                % left side
end
if nr > 0
    y(mo:mt, nt+1:ny) = x(:, 1:nr);              % right side
end

function demo_l2_l0
% demo_l2_l0 - This demo illustrates  the TwIST algorithm in
% the l2-l0 optimization problem
%
%     xe = arg min 0.5*||A x-y||^2 + tau ||x||_0
%           x
%
% which appears, for example, in compressed senseng (CS)
% applications
%
% For further details about the TwIST algorithm, see the paper:
%
% J. Bioucas-Dias and M. Figueiredo, "A New TwIST: Two-Step
% Iterative Shrinkage/Thresholding Algorithms for Image
% Restoration",  IEEE Transactions on Image processing, 2007.
%
%%
% Please check for the latest version of the code and papers at
% www.lx.it.pt/~bioucas/TwIST
%
% Authors: Jose Bioucas-Dias and Mario Figueiredo,
% Instituto Superior Técnico, October, 2007
%
```

```
%

% signal length
n = 4096;
% observation length
k = 1024;

% number of spikes
n_spikes = 100;
% random +/- 1 signal
x = zeros(n,1);
q = randperm(n);
x(q(1:n_spikes)) = sign(randn(n_spikes,1));

% measurement matrix
disp('Building measurement matrix...');
R = randn(k,n);
%normalize R
%  maxSingValue=svds(R,1);
%  R=R/maxSingValue;
% R has been precomputed
% load CSmatrix R
disp('Finished creating matrix');

%TwIST handles
% Linear operator handles
hR = @(x) R*x;
hRt = @(x) R'*x;
% define the regularizer and the respective denoising function
Psi = @(x,th) hard(x,th);   % denoising function
Phi = @(x) l0norm(x);       % regularizer

% noise variance
sigma=  1e-2;
% observed data
y = hR(x)+sigma*randn(k,1);

%  regularization parameter
tau = 20;

% stopping theshold
tolA = 1e-3;

% -- TwIST -------------------------
% stop criterium:  the relative change in the objective function
% falls below tolA
[x_twist,x_debias_twist,obj_twist,...
    times_twist,debias_start_twist,mse]= ...
        TwIST(y,hR,tau,...
        'Psi',Psi,...
        'Phi',Phi,...
        'AT', hRt, ...
        'Initialization',0,...
        'Monotone', 1, ...
        'StopCriterion',1,...
                        'ToleranceA',tolA,...
        'Verbose', 1);



figure(2)
subplot(1,2,1)
semilogy(times_twist,obj_twist,'b','LineWidth',2)
legend('TwIST')
st=sprintf('\\lambda = %2.2e, sigma = %2.2e',tau,sigma),...
title(st)
ylabel('Obj. function')
xlabel('CPU time (sec)')
grid
subplot(1,2,2)
plot(1:n,x_twist,'b', 1:n, x+2.5,'k','LineWidth',2)
axis([1 n -1.5 4 ]);
legend('TwIST', 'x')
st=sprintf('TwIST MSE = %2.2e', sum((x_twist-x).^2)/prod(size(x)));

title(st)

fprintf(1,'TwIST    MSE = %e\n', sum((x_twist-x).^2)/prod(size(x)));
fprintf(1,'TwIST   CPU time - %f\n', times_twist(end));



function y = masked_FFT(x,mask)
%
% Computes a 2D partial FFT transform,
% only for those frequencies for which
% mask is not zero. The result is returned
% in an array of size (k,1) where k is
% the number of non-zeros in mask.
% The transpose of this operator is
% available in the function masked_FFT_t.m
%
% Copyright, 2006, Mario Figueiredo.
%
%
[n1 n2] = size(x);
if n1~=n2
   disp('only square images, please!')
   return
end
n = n1;
if floor(log2(n))~=log2(n)
   disp('only power of 2 sizes, please!')
   return
end
if sum(sum(size(mask)~=size(x)))~=0
   disp('Mask must have the same size as x');
   return
end
ii = find(abs(mask)>0);
Rf = fftshift(fft2(x)).*mask/n;
y = Rf(ii);


function u = tvdenoise(f,lambda,iters)
```

```
%TVDENOISE  Total variation grayscale and color image denoising
%   u = TVDENOISE(f,lambda) denoises the input image f.  The smaller
%   the parameter lambda, the stronger the denoising.
%
%   The output u approximately minimizes the Rudin-Osher-Fatemi (ROF)
%   denoising model
%
%       Min  TV(u) + lambda/2 || f - u ||^2_2,
%        u
%
%   where TV(u) is the total variation of u.  If f is a color image (or any
%   array where size(f,3) > 1), the vectorial TV model is used,
%
%       Min  VTV(u) + lambda/2 || f - u ||^2_2.
%        u
%
%   TVDENOISE(...,Tol) specifies the stopping tolerance (default 1e-2).
%
%   The minimization is solved using Chambolle's method,
%      A. Chambolle, "An Algorithm for Total Variation Minimization and
%      Applications," J. Math. Imaging and Vision 20 (1-2): 89-97, 2004.
%   When f is a color image, the minimization is solved by a generalization
%   of Chambolle's method,
%      X. Bresson and T.F. Chan,  "Fast Minimization of the Vectorial Total
%      Variation Norm and Applications to Color Image Processing", UCLA CAM
%      Report 07-25.
%
%   Example:
%   f = double(imread('barbara-color.png'))/255;
%   f = f + randn(size(f))*16/255;
%   u = tvdenoise(f,12);
%   subplot(1,2,1); imshow(f); title Input
%   subplot(1,2,2); imshow(u); title Denoised

% Pascal Getreuer 2007-2008
%  Modified by Jose Bioucas-Dias  & Mario Figueiredo 2010
%  (stopping rule: iters)
%

if nargin < 3
    Tol = 1e-2;
end

if lambda < 0
    error('Parameter lambda must be nonnegative.');
end

dt = 0.25;

N = size(f);
id = [2:N(1),N(1)];
iu = [1,1:N(1)-1];
ir = [2:N(2),N(2)];
il = [1,1:N(2)-1];
p1 = zeros(size(f));
p2 = zeros(size(f));
divp = zeros(size(f));
lastdivp = ones(size(f));

if length(N) == 2           % TV denoising
    %while norm(divp(:) - lastdivp(:),inf) > Tol
    for i=1:iters
        lastdivp = divp;
        z = divp - f*lambda;
        z1 = z(:,ir) - z;
        z2 = z(id,:) - z;
        denom = 1 + dt*sqrt(z1.^2 + z2.^2);
        p1 = (p1 + dt*z1)./denom;
        p2 = (p2 + dt*z2)./denom;
        divp = p1 - p1(:,il) + p2 - p2(iu,:);
    end
elseif length(N) == 3       % Vectorial TV denoising
    repchannel = ones(N(3),1);

    %while norm(divp(:) - lastdivp(:),inf) > Tol
    for i=1:iters
        lastdivp = divp;
        z = divp - f*lambda;
        z1 = z(:,ir,:) - z;
        z2 = z(id,:,:) - z;
        denom = 1 + dt*sqrt(sum(z1.^2 + z2.^2,3));
        denom = denom(:,:,repchannel);
        p1 = (p1 + dt*z1)./denom;
        p2 = (p2 + dt*z2)./denom;
        divp = p1 - p1(:,il,:) + p2 - p2(iu,:,:);
    end
end

u = f – divp/lambda;

function y = TVnorm(x)
y = sum(sum(sqrt(diffh(x).^2+diffv(x).^2)));
```

SALSA basic function

```
% File: SALSA.m
%
% Usage:
% [x_estimate, objective, distance, times, mses] = SALSA(y,A,tau,varargin)
%
% This function solves the regularization problem
%
%     arg min_x = 0.5*|| y - A x ||_2^2 + tau phi( x ),
%
% where A is a generic matrix and phi(.) is a regularizarion
% function  such that the solution of the denoising problem
%
%     Psi_tau(y) = arg min_x = 0.5*|| y - x ||_2^2 + tau \phi( x ),
%
% is known.
%
% ----------------------------------------------------------------------
%
% For further details about the SALSA and C-SALSA algorithms, see the papers:
%
% [1] M. Afonso, J. Bioucas-Dias and M. Figueiredo. "Fast Algorithm for the Constrained
% Formulation of Compressive Image Reconstruction and Other Linear Inverse Problems",
% Submitted to IEEE International Conf. on Acoustics, Speech, and Signal Processing
% (ICASSP), Dallas, U.S.A., 2010.
% http://arxiv.org/abs/0909.3947v1
%
% [2] M. Afonso, J. Bioucas-Dias and M. Figueiredo. "Fast Image Recovery Using Variable
% Splitting and Constrained Optimization", IEEE Transactions on Image Processing,
% (Submitted), 2009.
% http://arxiv.org/abs/0910.4887
%
% [3] M. Figueiredo and J. Bioucas-Dias and M. Afonso. "Fast Frame-Based Image Decon-
% volution Using Variable Splitting and Constrained Optimization", IEEE Worskhop on
% Statistical Signal Processing, Cardiff, Wales, 2009.
% http://arxiv.org/abs/0904.4872
%
% ----------------------------------------------------------------------
% Authors: Manya Afonso, Jose Bioucas-Dias and Mario Figueiredo
% Date: October, 2009.
%
% For any technical queries/comments/bug reports, please contact:
% manya (dot) afonso (at) gmail (dot) com
% ----------------------------------------------------------------------
%
% Copyright (2009): Manya Afonso, Jose Bioucas-Dias, and Mario Figueiredo
%
% SALSA is distributed under the terms of
% the GNU General Public License 2.0.
%
% Permission to use, copy, modify, and distribute this software for
% any purpose without fee is hereby granted, provided that this entire
% notice is included in all copies of any software which is or includes
% a copy or modification of this software and in all copies of the
% supporting documentation for such software.
% This software is being provided "as is", without any express or
% implied warranty.  In particular, the authors do not make any
% representation or warranty of any kind concerning the merchantability
% of this software or its fitness for any particular purpose."
% ----------------------------------------------------------------------
%
%  ===== Required inputs =============
%
%  y: 1D vector or 2D array (image) of observations
%
%  A: if y and x are both 1D vectors, A can be a
%     k*n (where k is the size of y and n the size of x)
%     matrix or a handle to a function that computes
%     products of the form A*v, for some vector v.
%     In any other case (if y and/or x are 2D arrays),
%     A can be passed as a handle to a function which computes
%     products of the form A*x; another handle to a function
%     AT which computes products of the form A'*y is also required
%     in this case. A can also be the convolution kernel, or mask.
%     The size of x is determined as the size
%     of the result of applying AT.
%
%  tau: regularization parameter, usually a non-negative real
%       parameter of the objective  function (see above).
%
%
%  ===== Optional inputs =============
%
% 'MU' = parameter mu (weight for constraint function as a result of
%        splitting); default = 1e-3;
%
%  'Psi' = denoising function handle; handle to denoising function
%          Default = soft threshold.
%
%  'Phi' = function handle to regularizer needed to compute the objective
%          function.
%          Default = ||x||_1
%
%  'AT'  = function handle for the function that implements
%          the multiplication by the conjugate of A, when A
%          is a function handle.
%          If A is an array, AT is ignored.
%
% 'BASIS' = Synthesis operator (inverse wavelet transform, in case of wavelet frames);
%           default = identity
```

```
%
% 'BASISTRANSPOSE' = Analysis operator (forward transform); default = identity
%
% 'MASK' = must be set to 1, to indicate that A is not a masking
%          operator, but the mask (in the spatial domain). Do not specify
%          AT or INVAAT or ATA when using this.
%
% 'CONVOLUTIONFILTER' = must be set to 1, to indicate that A is not a
%          convolution operator, but the kernel (of the same size as the image).
%          Do not specify AT or INVAAT or ATA when using this.
%
% 'ATA' = function handle or matrix for AT(A(x)). May be needed when specified A
%          and AT are operators, especially when the observation is in
%          some transform domain. Must be used in combination with INVAAT.
%
% 'INVAAT' = term (mu I + A A^T)^(-1) appearing in the inverse of
%          (mu I + A^T A), throught the matrix inversion lemma. Must be a
%          scalar or matrix, to allow multiplication with the result of AT
%          or ATA.
%
% 'STOPCRITERION' = type of stopping criterion to use
%                 1 = stop when the relative
%                     change in the objective function
%                     falls below 'ToleranceA'
%                 2 = stop when the relative norm of the difference between
%                     two consecutive estimates falls below toleranceA
%                 3 = stop when the objective function
%                     becomes equal or less than toleranceA.
%                 Default = 1.
%
%  'TOLERANCEA' = stopping threshold; Default = 0.001
%
%  'MAXITERA' = maximum number of iterations allowed in the
%              main phase of the algorithm.
%              Default = 10000
%
%  'INNERITERS' = number of inner loop iterations; default = 1
%
%  'INITIALIZATION' must be one of {0,1,2,array}
%              0 -> Initialization at zero.
%              1 -> Random initialization.
%              2 -> initialization with A'*y.
%              array -> initialization provided by the user.
%              Default = 0;
%
%  'TRUE_X' = if the true underlying x is passed in
%               this argument, MSE evolution is computed
%
%  'ANALYSIS' if non-zero and wavelet basis specified, solves
%              min_x 0.5||A x - y ||^2 + tau *phi( W^T(x) )
%              default = 0 (synthesis prior)
%
%  'VERBOSE'  = work silently (0) or verbosely (1), default = 1
%
% 'OUTPUTVARIABLE' = which of the variables to return
%              min 0.5 ||Ax -y ||^2 + tau phi(z)
%              x,z
%                 s.t. x = z
%
%              1 (default); returns x
%              2 returns z
%
%
% ====================================================
% ============ Outputs ============================
%   x_estimate = solution of the main algorithm
%
%   objective = sequence of values of the objective function
%              0.5*|| y - A x ||_2^2 + tau phi( x )
%
%   distance = sequence of values of the constraint function
%
%   times = CPU time after each iteration
%
%   mses = sequence of MSE values, with respect to True_x,
%          if it was given; if it was not given, mses is empty,
%          mses = [].
% ======================================================

function [x_estimate, objective, distance, times, mses] = ...
        SALSA(y,A,tau,varargin)

%-----------------------------------------------------------------
% test for number of required parametres
%-----------------------------------------------------------------
if (nargin-length(varargin)) ~= 3
    error('Wrong number of required parameters');
end

%-----------------------------------------------------------------
% Set the defaults for the optional parameters
%-----------------------------------------------------------------
stopCriterion = 1;
compute_mse = 0;
maxiter = 10000; % outer loop iterations
init = 0;
AT = 0;
mu = 1e-3;
inneriters = 1;
psi_ok = 0;
tolA = 0.001;
isAconvfilter = 0;
isAmatrix = 0;
isAmask = 0;
verbose = 1;
definedbasis = 0;
definedbasistranspose = 0;
outputvar = 1;
isAT = 0;
isATA = 0;
isinvAAT = 0;

%-----------------------------------------------------------------
% Read the optional parameters
%-----------------------------------------------------------------
```

```
if (rem(length(varargin),2)==1)
  error('Optional parameters should always go by pairs');
else
  for i=1:2:(length(varargin)-1)
    switch upper(varargin{i})
     case 'BASIS'
        definedbasis = 1;
       W = varargin{i+1};
     case 'BASISTRANSPOSE'
        definedbasistranspose = 1;
       WT = varargin{i+1};
     case 'MASK'
        isAmask = varargin{i+1};
     case 'CONVOLUTIONFILTER'
       isAconvfilter = varargin{i+1};
     case 'PSI'
       psi_function = varargin{i+1};
     case 'PHI'
       phi_function = varargin{i+1};
     case 'MU'
       mu = varargin{i+1};
     case 'STOPCRITERION'
       stopCriterion = varargin{i+1};
     case 'TOLERANCEA'
       tolA = varargin{i+1};
     case 'INNERITERS'
        inneriters = varargin{i+1};
     case 'MAXITERA'
       maxiter = varargin{i+1};
     case 'INITIALIZATION'
       if numel(varargin{i+1}) > 1   % we have an initial x
                        init = 33333;    % some flag to be used below
                        x = varargin{i+1};
       else
                        init = varargin{i+1};
       end
     case 'TRUE_X'
       compute_mse = 1;
       true = varargin{i+1};
     case 'AT'
        isAT = 1;
       AT = varargin{i+1};
     case 'VERBOSE'
        verbose = varargin{i+1};
     case 'OUTPUTVARIABLE'
        outputvar = varargin{i+1};
     case 'ATA'
        ATA = varargin{i+1};
        isATA = 1;
     case 'INVAAT'
        invAAT = varargin{i+1};
        isinvAAT = 1;
     otherwise
      % Hmmm, something wrong with the parameter string
      error(['Unrecognized option: '' varargin{i} '']);
    end;
  end;
end
%%%%%%%%%%%%%%%

if (sum(stopCriterion == [1 2 3])==0)
   error(['Unknown stopping criterion']);
end

%%%% if the basis W was given, make sure WT is also given, and that their
%%%% dimensions are compatible.
if xor( definedbasis,definedbasistranspose )
    error(['If you give W you must also give WT, and vice versa.']);
end

if (~definedbasis)
    W = @(x) x;
    WT = @(x) x;
end


% if A is a function handle, we have to check presence of AT,
if isa(A, 'function_handle') && ~isa(AT,'function_handle')
   error(['The function handle for transpose of A is missing']);
end

% if A is a convolution filter, we have to generate handles to the
% operators A and AT
if (isAconvfilter)
    h = A;
    H_FFT = fft2(h);
    HC_FFT = conj(H_FFT);

    B = @(x) real(ifft2(H_FFT.*fft2(x)));
    BT = @(x) real(ifft2(HC_FFT.*fft2(x)));

end

% if A is a mask, we need to generate the handles for A and AT
if (isAmask)
    mask = A;
    B = @(x) mask.*x;
    BT = @(x) mask.*x;
end

% if A is a matrix, we find out dimensions of y and x,
% and create function handles for multiplication by A and A',
% so that the code below doesn't have to distinguish between
% the handle/not-handle cases
if ~( isa(A, 'function_handle') || isAconvfilter  || isAmask )
    isAmatrix = 1;

    if ~(size(y,1) == size(A,1) && size(y,2) == 1 )
        error('For a MxN observation matrix A, the measurement vector y must be a length M vector.');
    end

    if compute_mse && ~(size(true,1) == size(A,2) && size(true,2) == 1 )
        fprintf('\nWARNING: For a MxN observation matrix A, the vector x_true must be a length N vector.\n');
        compute_mse = 0;
    end
```

```
    matA = A;
    matAT = A';
   BT = @(x) matAT*x;
   B = @(x) matA*x;
end

if ( definedbasis )
        A = @(x) B(W(x));
        AT = @(x) WT(BT(x));
else
    if (~isAT)
        A = B;
        AT = BT;
    end
end
% from this point down, A and AT are always function handles.

% Precompute A'*y since it'll be used a lot
ATy = AT(y);

%%%%% verify if ATA is compatible with A and AT;
if (isATA)
    dummy  = ATy;
    ATAd = AT(A(dummy));
    if (ATAd ~= ATA(dummy))
        error('Specified ATA does not seem compatible with the specified A and AT.\n')
    end
end

%%%% verify if invAAT has dimensions compatible with A and AT
if(isinvAAT)
    if isa(invAAT, 'function_handle')
        error('Specified invAAT must be a scalar or a matrix of compatible dimensions.\n')
    else
        if (numel(invAAT)>1)
            dummy = ATy;
            dummy_prod = AT(invAAT*A(dummy));
            if ~((size(dummy,1)==size(dummy_prod,1))&&(size(dummy,2)==size(dummy_prod,2)))
                error('Specified invAAT does not seem compatible with the specified A and AT.\n')
            end
        end
    end
end

% if phi was given, check to see if it is a handle and that it
% accepts two arguments
if exist('psi_function','var')
   if isa(psi_function,'function_handle')
      try  % check if phi can be used, using Aty, which we know has
           % same size as x
            dummy = psi_function(ATy,tau);
            psi_ok = 1;
      catch
         error(['Something is wrong with function handle for psi'])
      end
   else
      error(['Psi does not seem to be a valid function handle']);
   end
else %if nothing was given, use soft thresholding
   psi_function = @(x,tau) soft(x,tau);
end

% if psi exists, phi must also exist
if (psi_ok == 1)
   if exist('phi_function','var')
      if isa(phi_function,'function_handle')
         try  % check if phi can be used, using Aty, which we know has
              % same size as x
              dummy = phi_function(ATy);
         catch
           error(['Something is wrong with function handle for phi'])
         end
      else
         error(['Phi does not seem to be a valid function handle']);
      end
   else
      error(['If you give Psi you must also give Phi']);
   end
else  % if no psi and phi were given, simply use the l1 norm.
   phi_function = @(x) sum(abs(x(:)));
end

%-----------------------------------------------------------------
% Initialization
%-----------------------------------------------------------------
switch init
    case 0   % initialize at zero, using AT to find the size of x
       x = AT(zeros(size(y)));
    case 1   % initialize randomly, using AT to find the size of x
       x = randn(size(AT(zeros(size(y)))));
    case 2   % initialize x0 = A'*y
       x = ATy;
    case 33333
       % initial x was given as a function argument; just check size
       if size(A(x)) ~= size(y)
          error(['Size of initial x is not compatible with A']);
       end
    otherwise
       error(['Unknown "Initialization" option']);
end

% if the true x was given, check its size
if compute_mse & (size(true) ~= size(x))
   error(['Initial x has incompatible size']);
end

% initializing
z = AT(zeros(size(y)));
b = z;
muinv = 1/mu;
threshold = tau/mu;
criterion(1) = 1;

% Compute and store initial value of the objective function
resid =  y-A(x);
```

```
prev_f = 0.5*(resid(:)'*resid(:)) + tau*phi_function(x);

if verbose
    fprintf('Initial value of objective function = %3.3g\n',prev_f)
end

% start the clock
t0 = cputime;

if (isAconvfilter)
    filter_FFT = HC_FFT./(abs(H_FFT).^2 + mu).*H_FFT;
end

if (isAmatrix && (~isinvAAT))
    AAT = matA*matAT;
    invAAT = inv(mu*eye(size(AAT))+AAT);
end

if (isAmask && (~isinvAAT))
    invAAT = 1./(mu+mask);
end

times(1) = 0;
objective(1) = prev_f;

if compute_mse
   mses(1) = sum(sum((x-true).^2))/numel(x);
end

global numA;

for outer = 1:maxiter

    for inner = 1:inneriters

        z = psi_function(x-b, threshold);
        r = ATy + mu*(z+b);

        if (isAconvfilter)
            x = muinv*( r - WT( real(ifft2(filter_FFT.*fft2( W(r) ))) ) );
            numA = numA + 1;
        else
            if (isATA)
                x = muinv*( r -  invAAT*ATA(r) );
            else
                if (isAmatrix)
                    x = muinv*(r - AT(invAAT*A(r)));
                else
                    if (isAmask)
                        x = muinv*(r - AT(invAAT.*A(r)));
                    end
                end
            end
        end

    end

    b = b + (z - x);

    resid =  y-A(x);
    objective(outer+1) = 0.5*(resid(:)'*resid(:)) + tau*phi_function(x);

    if compute_mse
        err = x-true;
        mses(outer+1) =  (err(:)'*err(:))/numel(x);
    end

    distance(outer) = norm(x(:)-z(:),2)/sqrt(norm(x(:),2)^2 + norm(z(:),2)^2);

    if (outer>1)
        % take no more than maxiter iterations
        switch stopCriterion
            case 1
                % compute the stopping criterion based on the relative
                % variation of the objective function.
                criterion(outer) = abs(objective(outer+1)-objective(outer))/objective(outer);
            case 2
                % compute the stopping criterion based on the relative
                % variation of the estimate.
                criterion(outer) = abs(norm(x(:)-xprev(:))/norm(x(:)));
            case 3
                % continue if not yet reached target value tolA
                criterion(outer) = objective(outer+1);
            otherwise
                error(['Unknown stopping criterion']);
        end

         if ( criterion(outer) < tolA )
            if verbose
                fprintf('\niter = %d, obj = %3.3g, ||x-z|| = %3.3g, stop criterion = %3.3g, ( target = %3.3g )\t', outer, objective(outer+1), distance(outer), criterion(outer), tolA)
                if compute_mse
                    fprintf('MSE = %g',mses(outer+1))
                end

                fprintf('\nConvergence reached for Bregman iterative procedure.\n')
            end
            times(outer+1) = cputime - t0;
            break;
         end

    end

    if verbose
        fprintf('\niter = %d, obj = %3.3g, ||x-z|| = %3.3g, stop criterion = %3.3g, ( target = %3.3g )\t', outer, objective(outer+1), distance(outer), criterion(outer), tolA)
        if compute_mse
            fprintf('MSE = %g',mses(outer+1))
        end
    end

    times(outer+1) = cputime - t0;
end

times(outer) = cputime - t0;

if (outputvar == 2)
```

```
    x_estimate = z;
else
    x_estimate = x;
end



% File: SALSA.m
%
% Usage:
% [x_estimate, numA, numAt, objective, distance, times, mses] = SALSA(y,A,tau,varargin)
%
% This function solves the regularization problem
%
%     arg min_x = 0.5*|| y - A x ||_2^2 + tau phi( P^T x ),
%
% where A is a generic linear operator and phi(.) is a convex regularizarion
% function  such that the solution of the denoising problem
%
%     Psi_tau(y) = arg min_x = 0.5*|| y - x ||_2^2 + tau \phi( x ),
%
% is known.
%
% ----------------------------------------------------------------------
%
% For further details about the SALSA and C-SALSA algorithms, see the papers:
%
% [1] M. Afonso, J. Bioucas-Dias, and M. Figueiredo, "Fast image recovery
% using variable splitting and constrained optimization," IEEE Transactions
% on Image Processing, vol. 19, no. 9, pp. 2345-2356, September, 2010.
%
% [2] M. Afonso, J. Bioucas-Dias, and M. Figueiredo, "An Augmented
% Lagrangian based Method for the Constrained Formulation of Imaging Inverse
% Problems", IEEE Transactions on Image Processing, Vol. 20, no. 3,
% pp 681 - 695, March, 2011.
%
% ----------------------------------------------------------------------
% Authors: Manya Afonso, Jose Bioucas-Dias and Mario Figueiredo
% Date: October, 2009.
% Revised: August, 2011
%
% For any technical queries/comments/bug reports, please contact:
% manya (dot) afonso (at) gmail (dot) com
% ----------------------------------------------------------------------
%
% Copyright (2009): Manya Afonso, Jose Bioucas-Dias, and Mario Figueiredo
%
% SALSA is distributed under the terms of
% the GNU General Public License 2.0.
%
% Permission to use, copy, modify, and distribute this software for
% any purpose without fee is hereby granted, provided that this entire
% notice is included in all copies of any software which is or includes
% a copy or modification of this software and in all copies of the
% supporting documentation for such software.
% This software is being provided "as is", without any express or
% implied warranty.  In particular, the authors do not make any
% representation or warranty of any kind concerning the merchantability
% of this software or its fitness for any particular purpose."
% ----------------------------------------------------------------------
%
%  ===== Required inputs =============
%
%  y: 1D vector or 2D array (image) of observations
%
%  A: if y and x are both 1D vectors, A can be a
%     k*n (where k is the size of y and n the size of x)
%     matrix or a handle to a function that computes
%     products of the form A*v, for some vector v.
%     In any other case (if y and/or x are 2D arrays),
%     A can be passed as a handle to a function which computes
%     products of the form A*x; another handle to a function
%     AT which computes products of the form A'*y is also required
%     in this case. A can also be the convolution kernel, or mask.
%     The size of x is determined as the size
%     of the result of applying AT.
%
%  tau: regularization parameter, usually a non-negative real
%       parameter of the objective  function (see above).
%
%
%  ===== Optional inputs =============
%
% 'MU' = parameter mu (weight for constraint function as a result of
%        splitting); default = 1e-3;
%
%  'Psi' = denoising function handle; handle to denoising function
%          Default = soft threshold.
%
%  'Phi' = function handle to regularizer needed to compute the objective
%          function.
%          Default = ||x||_1
%
% 'TVINITIALIZATION' = must be set to 1 if using TV with initialization of
% the dual variables. Phi and Psi are ignored, if specified.
%
% 'TViters' = number of iterations of Chambolle's algorithm, if using TV with initialization of
% the dual variables. Default = 5. Ignored if 'TVINITIALIZATION' is 0 or unspecified.
%
%  'AT'  = function handle for the function that implements
%          the multiplication by the conjugate of A, when A
%          is a function handle.
%          If A is an array, AT is ignored.
%
% 'PT' = Transform in analysis prior formulation (inverse wavelet transform, in case of wavelet frames);
%          default = identity
%
% 'P' = inverse transform in analysis prior formulation (forward transform); default = identity
%
% 'LS' = function handle for multiplication with the term (mu I + A^T A)^(-1)
%          Can be omitted if A is a matrix.
%
% 'STOPCRITERION' = type of stopping criterion to use
%                   1 = stop when the relative
%                       change in the objective function
```

```
%                    falls below 'ToleranceA'
%                 2 = stop when the relative norm of the difference between
%                    two consecutive estimates falls below toleranceA
%                 3 = stop when the objective function
%                    becomes equal or less than toleranceA.
%                 Default = 1.
%
%  'TOLERANCEA' = stopping threshold; Default = 0.001
%
%  'MAXITERA' = maximum number of iterations allowed in the
%              main phase of the algorithm.
%              Default = 10000
%
%  'INITIALIZATION' must be one of {0,1,2,array}
%              0 -> Initialization at zero.
%              1 -> Random initialization.
%              2 -> initialization with A'*y.
%              array -> initialization provided by the user.
%              Default = 0;
%
%  'TRUE_X' = if the true underlying x is passed in
%               this argument, MSE evolution is computed
%
%  'VERBOSE'  = work silently (0) or verbosely (1), default = 1
%

%
% ====================================================
% ============ Outputs ============================
%   x_estimate = solution of the main algorithm
%
%   numA, numAt = number of calls to A and At (if A is a matrix)
%          NOTE: if A,AT are function handles, the wrapper callcounter()
%          must be used to increment the global variable 'calls'
%
%   objective = sequence of values of the objective function
%              0.5*|| y - A x ||_2^2 + tau phi( x )
%
%   distance = sequence of values of the constraint function
%
%   times = CPU time after each iteration
%
%   mses = sequence of MSE values, with respect to True_x,
%         if it was given; if it was not given, mses is empty,
%         mses = [].
% ========================================================

function [x, numA, numAt, objective, distance, times, mses] = ...
        SALSA_v2(y,A,tau,varargin)

%-----------------------------------------------------------------
% test for number of required parametres
%-----------------------------------------------------------------
% if (nargin-length(varargin)) ~= 3
%      error('Wrong number of required parameters');
% end

%-----------------------------------------------------------------
% Set the defaults for the optional parameters
%-----------------------------------------------------------------
stopCriterion = 1;
compute_mse = 0;
maxiter = 10000; % outer loop iterations
init = 0;
AT = 0;

mu = 1e-3;

psi_ok = 0;
tolA = 0.001;
isTVinitialization = 0;
TViters = 5;
verbose = 1;

definedP = 0;
definedPT = 0;

isinvLS = 0;
numA = 0;
numAt = 0;

%-----------------------------------------------------------------
% Read the optional parameters
%-----------------------------------------------------------------
if (rem(length(varargin),2)==1)
  error('Optional parameters should always go by pairs');
else
  for i=1:2:(length(varargin)-1)
    switch upper(varargin{i})
     case 'P'
        definedP = 1;
      P = varargin{i+1};
     case 'PT'
        definedPT = 1;
      PT = varargin{i+1};
     case 'PSI'
      psi = varargin{i+1};
     case 'PHI'
      phi = varargin{i+1};
     case 'TVINITIALIZATION'
       isTVinitialization = varargin{i+1};
     case 'TVITERS'
       TViters = varargin{i+1};
                         case 'MU'
      mu = varargin{i+1};
     case 'STOPCRITERION'
      stopCriterion = varargin{i+1};
     case 'TOLERANCEA'
      tolA = varargin{i+1};
     case 'MAXITERA'
      maxiter = varargin{i+1};
     case 'INITIALIZATION'
      if numel(varargin{i+1}) > 1   % we have an initial x
                         init = 33333;    % some flag to be used below
                         x = varargin{i+1};
```

```
      else
                        init = varargin{i+1};
      end
     case 'TRUE_X'
      compute_mse = 1;
      true = varargin{i+1};
     case 'AT'
        AT = varargin{i+1};
     case 'VERBOSE'
        verbose = varargin{i+1};
     case 'LS'
        invLS = varargin{i+1};
        isinvLS = 1;
     otherwise
      % Hmmm, something wrong with the parameter string
      error(['Unrecognized option: "' varargin{i} '"']);
    end;
  end;
end
%%%%%%%%%%%%%%

if (sum(stopCriterion == [1 2 3])==0)
   error(['Unknown stopping criterion']);
end

%%%% if P was given, make sure PT is also given, and that their
%%%% dimensions are compatible.
if xor( definedP,definedPT )
    error(['If you give P you must also give PT, and vice versa.']);
end

if (~definedP)
    P = @(x) x;
    PT = @(x) x;
end

% if A is a function handle, we have to check presence of AT,
if isa(A, 'function_handle') && ~isa(AT,'function_handle')
   error(['The function handle for transpose of A is missing']);
end

% if A is a matrix, we find out dimensions of y and x,
% and create function handles for multiplication by A and A',
% so that the code below doesn't have to distinguish between
% the handle/not-handle cases
if ~( isa(A, 'function_handle') )

    if ~(size(y,1) == size(A,1) && size(y,2) == 1 )
        error('For a MxN observation matrix A, the measurement vector y must be a length M vector.');
    end

    if compute_mse && ~(size(true,1) == size(A,2) && size(true,2) == 1 )
        fprintf('\nWARNING: For a MxN observation matrix A, the vector x_true must be a length N vector.\n');
        compute_mse = 0;
    end
    matA = A;
    matAT = A';
   AT = @(x) matAT*x;
   A = @(x) matA*x;
end

% from this point down, A and AT are always function handles.

% Precompute A'*y since it'll be used a lot
ATy = AT(y);
numAt = numAt + 1;


% if A is a function handle, we have to check presence of invLS,
if isa(A, 'function_handle')

    if ( ~isinvLS || ~isa(invLS, 'function_handle'))
        error(['(A^T A + \mu I)^(-1) must be specified as a function handle.\n']);
    else
        dummy = invLS(ATy);
        if ~((size(dummy,1)==size(ATy,1))&&(size(dummy,2)==size(ATy,2)))
            error('Specified function handle for solving the LS step does not seem compatible with the specified A and AT.\n')
        end
    end
else %% A is a matrix
    [M,N] = size(matA);
    if ( M > N )
        ATA = transpose(matA)*matA;
        inverse_term = inv(ATA+mu*eye(N,N));
        %inverse_term = @(x) invATA*x;
    else
        AAT = matA*transpose(matA);
        inverse_term = AT*inv(mu*eye(M,M)+AAT)*A;
    end
    invLS = @(x) inverse_term*x;
end

% if psi was given, check to see if it is a handle and that it
% accepts two arguments
if exist('psi','var') %& ~isTVinitialization
    if isTVinitialization
        fprintf('Warning: user specified Phi and Psi will not be used as TV with initialization flag has been set to 1.\n')
    else
       if isa(psi,'function_handle')
          try  % check if phi can be used, using Aty, which we know has
               % same size as x
               dummy = psi(PT(ATy),tau);
               psi_ok = 1;
          catch
            error(['Something is wrong with function handle for psi'])
          end
       else
          error(['Psi1 does not seem to be a valid function handle']);
       end
    end
else %if nothing was given, use soft thresholding
   psi = @(x,tau) soft(x,tau);
end

% if psi exists, phi must also exist
```

```
if (psi_ok == 1)
   if exist('phi','var')
      if isa(phi,'function_handle')
         try  % check if phi can be used, using Aty, which we know has
              % same size as x
              dummy = phi( PT(ATy) );
         catch
           error(['Something is wrong with function handle for phi'])
         end
      else
       error(['Phi does not seem to be a valid function handle']);
      end
   else
      error(['If you give Psi you must also give Phi']);
   end
else  % if no psi and phi were given, simply use the l1 norm.
   if ~isTVinitialization
      phi = @(x) sum(abs(x(:)));
   else
      phi = @(x) TVnorm(x);
   end
end



%-----------------------------------------------------------------
% Initialization
%-----------------------------------------------------------------
switch init
    case 0   % initialize at zero, using AT to find the size of x
       x = AT(zeros(size(y)));
    case 1   % initialize randomly, using AT to find the size of x
       x = randn(size(AT(zeros(size(y)))));
    case 2   % initialize x0 = A'*y
       x = ATy;
    case 33333
       % initial x was given as a function argument; just check size
       if size(A(x)) ~= size(y)
          error(['Size of initial x is not compatible with A']);
       end
    otherwise
       error(['Unknown "Initialization" option']);
end

% if the true x was given, check its size
if compute_mse & (size(true) ~= size(x))
   error(['Initial x has incompatible size']);
end


PTx = PT(x);

% initializing
u = PTx;
bu = 0*u;
threshold = tau/mu;

criterion(1) = 1;

% Compute and store initial value of the objective function
resid =  y-A(x);
numA = numA + 1;
prev_f = 0.5*(resid(:)'*resid(:)) + tau*phi(u);

if verbose
    fprintf('Initial value of objective function = %3.3g\n',prev_f)
end

% start the clock
t0 = cputime;


times(1) = 0;
objective(1) = prev_f;

if compute_mse
   mses(1) = sum(sum((x-true).^2))/numel(x);
end

if isTVinitialization
    pux = 0*u;
    puy = 0*u;
end

for outer = 1:maxiter

    xprev = x;


        if isTVinitialization
              [u,pux,puy] = chambolle_prox_TV_stop(real(PTx-bu), 'lambda', threshold, 'maxiter', TViters, 'dualvars',[pux puy]);
        else
            u = psi(PTx-bu, threshold);
        end

        r = ATy + mu*P(u+bu);

        x = invLS(r);

    PTx = PT(x);

    bu = bu + (u - PTx);

    resid =  y-A(x);
    numA = numA + 1;
    objective(outer+1) = 0.5*(resid(:)'*resid(:)) + tau*phi(u);

    if compute_mse
        err = x-true;
        mses(outer+1) =  (err(:)'*err(:))/numel(x);
    end

    distance(outer) = norm(PTx(:)-u(:),2)/sqrt(norm(PTx(:),2)^2 + norm(u(:),2)^2);

    if (outer>1)
```

```
        % take no more than maxiter iterations
        switch stopCriterion
            case 1
                % compute the stopping criterion based on the relative
                % variation of the objective function.
                criterion(outer) = abs(objective(outer+1)-objective(outer))/objective(outer);
            case 2
                % compute the stopping criterion based on the relative
                % variation of the estimate.
                criterion(outer) = abs(norm(x(:)-xprev(:))/norm(x(:)));
            case 3
                % continue if not yet reached target value tolA
                criterion(outer) = objective(outer+1);
            otherwise
                error(['Unknown stopping criterion']);
        end

        if ( criterion(outer) < tolA )
            if verbose
                fprintf('\niter = %d, obj = %3.3g, stop criterion = %3.3g, ( target = %3.3g )\t', outer, objective(outer+1), criterion(outer), tolA)
                if compute_mse
                    fprintf('MSE = %g',mses(outer+1))
                end

                fprintf('\nConvergence reached.\n')
            end
            times(outer+1) = cputime - t0;
            break;
        end

    end

    if verbose
        fprintf('\niter = %d, obj = %3.3g, stop criterion = %3.3g, ( target = %3.3g )\t', outer, objective(outer+1), criterion(outer), tolA)
        if compute_mse
            fprintf('MSE = %g',mses(outer+1))
        end
    end

    times(outer+1) = cputime - t0;
end



function y = masked_FFT_t(x,mask)
%
% This is the transpose operator of the
% partial FFT transform, implemented in
% masked_FFT.m.  See that file for details.
%
[n1,n2] = size(mask);
if n1~=n2
   disp('only square images please!')
   return
end
n = n1;
if floor(log2(n))~=log2(n)
   disp('only power of 2 sizes, please!')
   return
end
ii=find(abs(mask)>0);
gg = zeros(n,n);
gg(ii)=x./mask(ii);
y = real(ifft2(ifftshift(gg)))*n;



function [h_0,h_1] = daubcqf(N,TYPE)
%    [h_0,h_1] = daubcqf(N,TYPE);
%
%    Function computes the Daubechies' scaling and wavelet filters
%    (normalized to sqrt(2)).
%
%    Input:
%       N    : Length of filter (must be even)
%       TYPE : Optional parameter that distinguishes the minimum phase,
%              maximum phase and mid-phase solutions ('min', 'max', or
%              'mid'). If no argument is specified, the minimum phase
%              solution is used.
%
%    Output:
%       h_0 : Minimal phase Daubechies' scaling filter
%       h_1 : Minimal phase Daubechies' wavelet filter
%
%    Example:
%       N = 4;
%       TYPE = 'min';
%       [h_0,h_1] = daubcqf(N,TYPE)
%       h_0 = 0.4830 0.8365 0.2241 -0.1294
%       h_1 = 0.1294 0.2241 -0.8365 0.4830
%
%    Reference: "Orthonormal Bases of Compactly Supported Wavelets",
%               CPAM, Oct.89
%
%Author: Ramesh Gopinath  <ramesh@dsp.rice.edu>

if(nargin < 2),
  TYPE = 'min';
end;
if(rem(N,2) ~= 0),
  error('No Daubechies filter exists for ODD length');
end;
K = N/2;
a = 1;
p = 1;
q = 1;
h_0 = [1 1];
for j  = 1:K-1,
  a = -a * 0.25 * (j + K - 1)/j;
  h_0 = [0 h_0] + [h_0 0];
  p = [0 -p] + [p 0];
  p = [0 -p] + [p 0];
  q = [0 q 0] + a*p;
end;
q = sort(roots(q));
```

```
qt = q(1:K-1);
if TYPE=='mid',
  if rem(K,2)==1,
    qt = q([1:4:N-2 2:4:N-2]);
  else
    qt = q([1 4:4:K-1 5:4:K-1 N-3:-4:K N-4:-4:K]);
  end;
end;
h_0 = conv(h_0,real(poly(qt)));
h_0 = sqrt(2)*h_0/sum(h_0);          %Normalize to sqrt(2);
if(TYPE=='max'),
  h_0 = fliplr(h_0);
end;
if(abs(sum(h_0 .^ 2))-1 > 1e-4)
  error('Numerically unstable for this value of "N".');
end;
h_1 = rot90(h_0,2);
h_1(1:2:N)=-h_1(1:2:N);



function [xd,xn,option] = denoise(x,h,type,option)
%    [xd,xn,option] = denoise(x,h,type,option);
%
%    DENOISE is a generic program for wavelet based denoising.
%    The program will denoise the signal x using the 2-band wavelet
%    system described by the filter h using either the traditional
%    discrete wavelet transform (DWT) or the linear shift invariant
%    discrete wavelet transform (also known as the undecimated DWT
%    (UDWT)).
%
%    Input:
%       x         : 1D or 2D signal to be denoised
%       h         : Scaling filter to be applied
%       type      : Type of transform (Default: type = 0)
%                   0 --> Discrete wavelet transform (DWT)
%                   1 --> Undecimated DWT (UDWT)
%       option    : Default settings is marked with '*':
%                   *type = 0 --> option = [0 3.0 0 0 0 0]
%                   type = 1 --> option = [0 3.6 0 1 0 0]
%       option(1) : Whether to threshold low-pass part
%                   0 --> Don't threshold low pass component
%                   1 --> Threshold low pass component
%       option(2) : Threshold multiplier, c. The threshold is
%                   computed as:
%                     thld = c*MAD(noise_estimate)).
%                   The default values are:
%                     c = 3.0 for the DWT based denoising
%                     c = 3.6 for the UDWT based denoising
%       option(3) : Type of variance estimator
%                   0 --> MAD (mean absolute deviation)
%                   1 --> STD (classical numerical std estimate)
%       option(4) : Type of thresholding
%                   2 --> Soft thresholding
%                   1 --> Hard thresholding
%       option(5) : Number of levels, L, in wavelet decomposition. By
%                   setting this to the default value '0' a maximal
%                   decomposition is used.
%       option(6) : Actual threshold to use (setting this to
%                   anything but 0 will mean that option(3)
%                   is ignored)
%
%    Output:
%       xd     : Estimate of noise free signal
%       xn     : The estimated noise signal (x-xd)
%       option : A vector of actual parameters used by the
%                program. The vector is configured the same way as
%                the input option vector with one added element
%                option(7) = type.
%
%  HERE'S AN EASY WAY TO RUN THE EXAMPLES:
%  Cut-and-paste the example you want to run to a new file
%  called ex.m, for example. Delete out the % at the beginning
%  of each line in ex.m (Can use search-and-replace in your editor
%  to replace it with a space). Type 'ex' in matlab and hit return.
%
%    Example 1:
%       h = daubcqf(6); [s,N] = makesig('Doppler'); n = randn(1,N);
%       x = s + n/10;     % (approximately 10dB SNR)
%       figure;plot(x);hold on;plot(s,'r');
%
%       %Denoise x with the default method based on the DWT
%       [xd,xn,opt1] = denoise(x,h);
%       figure;plot(xd);hold on;plot(s,'r');
%
%       %Denoise x using the undecimated (LSI) wavelet transform
%       [yd,yn,opt2] = denoise(x,h,1);
%       figure;plot(yd);hold on;plot(s,'r');
%
% Example 2: (on an image)
%      h = daubcqf(6);  load lena;
%      noisyLena = lena + 25 * randn(size(lena));
%      figure; colormap(gray); imagesc(lena); title('Original Image');
%       figure; colormap(gray); imagesc(noisyLena); title('Noisy Image');
%       Denoise lena with the default method based on the DWT
%      [denoisedLena,xn,opt1] = denoise(noisyLena,h);
%      figure; colormap(gray); imagesc(denoisedLena); title('denoised Image');
%
%
%    See also: mdwt, midwt, mrdwt, mirdwt, SoftTh, HardTh, setopt
%
%Author: Jan Erik Odegard  <odegard@ece.rice.edu>

if(nargin < 2)
  error('You need to provide at least 2 inputs: x and h');
end;
if(nargin < 3),
  type = 0;
  option = [];
elseif(nargin < 4)
  option = [];
end;
if(isempty(type)),
  type = 0;
end;
```

```
if(type == 0),
  default_opt = [0 3.0 0 2 0 0];
elseif(type == 1),
  default_opt = [0 3.6 0 1 0 0];
else
  error(['Unknown denoising method',10,...
                    'If it is any good we need to have a serious talk :-)']);
end;
option = setopt(option,default_opt);
[mx,nx] = size(x);
dim = min(mx,nx);
if(dim == 1),
  n = max(mx,nx);
else
  n = dim;
end;
if(option(5) == 0),
  L = floor(log2(n));
else
  L = option(5);
end;
if(type == 0),                                        % Denoising by DWT
  xd = mdwt(x,h,L);
  if (option(6) == 0),
    tmp = xd(floor(mx/2)+1:mx,floor(nx/2)+1:nx);
    if(option(3) == 0),
      thld = option(2)*median(abs(tmp(:)))/.67;
    elseif(option(3) == 1),
      thld = option(2)*std(tmp(:));
    else
      error('Unknown threshold estimator, Use either MAD or STD');
    end;
  else
    thld = option(6);
  end;
  if(dim == 1)
    ix = 1:n/(2^L);
    ykeep = xd(ix);
  else
    ix = 1:mx/(2^L);
    jx = 1:nx/(2^L);
    ykeep = xd(ix,jx);
  end;
  if(option(4) == 2),
    xd = SoftTh(xd,thld);
  elseif(option(4) == 1),
    xd = HardTh(xd,thld);
  else
    error('Unknown threshold rule. Use either Soft (2) or Hard (1)');
  end;
  if (option(1) == 0),
    if(dim == 1),
      xd(ix) = ykeep;
    else
      xd(ix,jx) = ykeep;
    end;
  end;
  xd = midwt(xd,h,L);
elseif(type == 1),                                    % Denoising by UDWT
  [xl,xh] = mrdwt(x,h,L);
  if(dim == 1),
    c_offset = 1;
  else
    c_offset = 2*nx + 1;
  end;
  if (option(6) == 0),
    tmp = xh(:,c_offset:c_offset+nx-1);
    if(option(3) == 0),
      thld = option(2)*median(abs(tmp(:)))/.67;
    elseif(option(3) == 1),
      thld = option(2)*std(tmp(:));
    else
      error('Unknown threshold estimator, Use either MAD or STD');
    end;
  else
    thld = option(6);
  end;
  if(option(4) == 2),
    xh = SoftTh(xh,thld);
    if(option(1) == 1),
      xl = SoftTh(xl,thld);
    end;
  elseif(option(4) == 1),
    xh = HardTh(xh,thld);
    if(option(1) == 1),
      xl = HardTh(xl,thld);
    end;
  else
    error('Unknown threshold rule. Use either Soft (2) or Hard (1)');
  end;
  xd = mirdwt(xl,xh,h,L);
else                                                  % Denoising by unknown method
  error(['Unknown denoising method',10,...
        'If it is any good we need to have a serious talk :-)']);
end;
option(6) = thld;
option(7) = type;
xn = x - xd;
```